LeanCode

Kontekst dla BuildContext

Tomek Koter

senior dev @ LeanCode

tomasz.koter@leancode.pl

Plan na dziś

- co kryje się pod spodem widgetów – trzy drzewa
- budowanie i przebudowa drzewa
- czym jest BuildContext
- interakcje – hit testing i gesty
- StatefulWidget – stan lokalny
- klucze
- InheritedWidget i Provider – dane w dół drzewa
- BuildContext w praktyce
- Listenable – reagowanie na zmiany
- dobre praktyki

We build digital products.

LeanCode

①	User input	Responses to input gestures (keyboard, touchscreen, etc.)
②	Animation	User interface changes triggered by the tick of a timer
③	Build	App code that creates widgets on the screen
④	Layout	Positioning and sizing elements on the screen
⑤	Paint	Converting elements into a visual representation
⑥	Composition	Overlaying visual elements in draw order
⑦	Rasterize	Translating output into GPU render instructions

RENDERING

Źródło: Flutter architectural overview

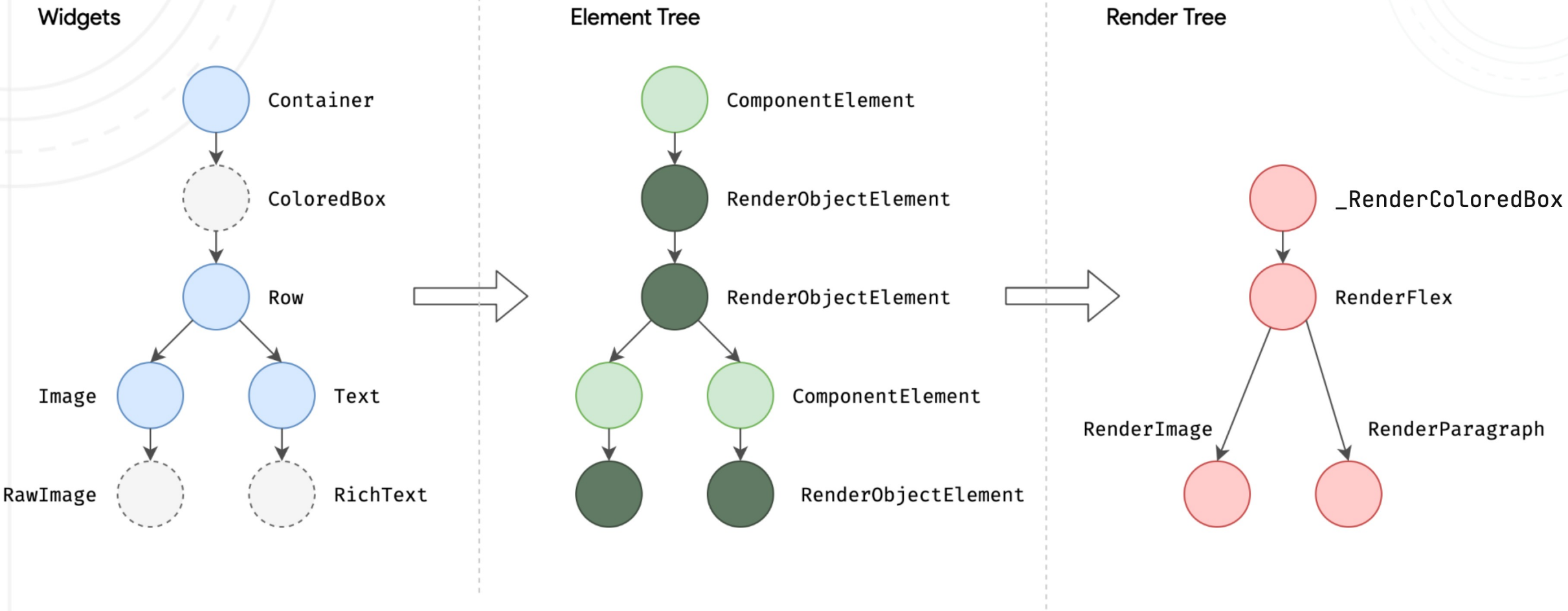
We build digital products

LeanCode

lekkie i niezmienne

trwałe i mutowalne

layout · paint · dostępność



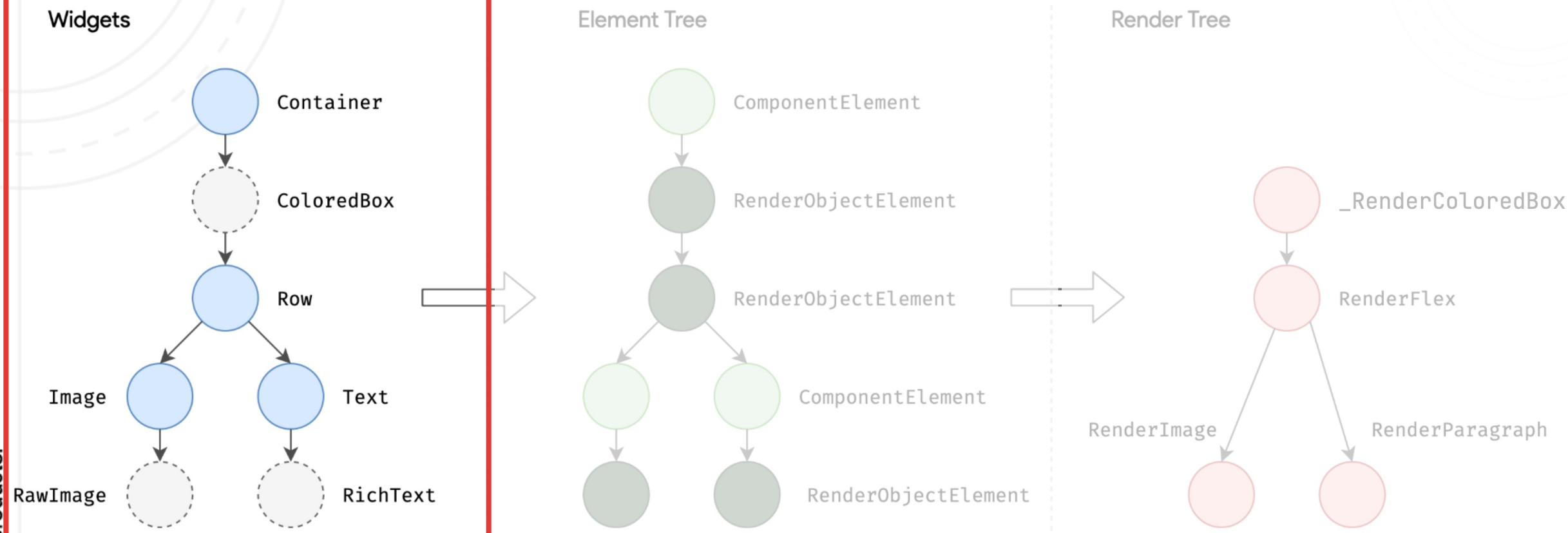
Źródło: https://docs.flutter.dev/resources/architectural-overview#layout-and-rendering

We build digital products.
LeanCode

lekkie i niezmienne

trwałe i mutowalne

layout · paint · dostępność



Źródło: https://docs.flutter.dev/resources/architectural-overview#layout-and-rendering

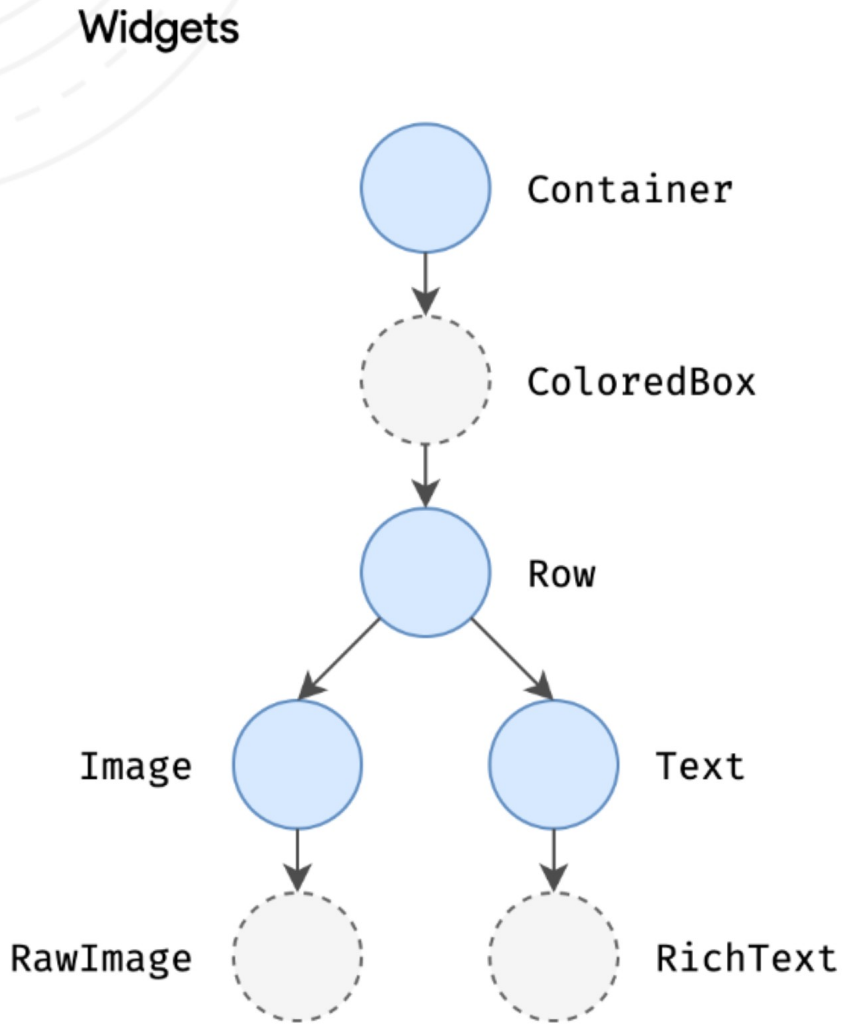
We build digital products.
LeanCode

```dart
class MyWidget extends StatelessWidget {
  const MyWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.amber.shade100,
      width: 100,
      height: 100,
      child: Row(
        children: [
          Image.network('https://picsum.photos/200/300'),
          Text('Hello World!'),
        ],
      ),
    );
  }
}
```

We build digital products.

LeanCode

Drzewo widgetów



widgety z naszego kodu

dobudowane w **build()** innych widgetów (Container, Image, Text)

We build digital products.

LeanCode

```
246  class Container extends StatelessWidget {
381    @override
382    Widget build(BuildContext context) {
383      Widget? current = child;
384
385      if (child == null && (constraints == null || !constraints!.isTight)) {
386        current = LimitedBox(
387          maxWidth: 0.0,
388          maxHeight: 0.0,
389          child: ConstrainedBox(constraints: const BoxConstraints.expand()),
390        );
391      } else if (alignment != null) {
392        current = Align(alignment: alignment!, child: current);
393      }
394
395      final EdgeInsetsGeometry? effectivePadding = _paddingIncludingDecoration;
396      if (effectivePadding != null) {
397        current = Padding(padding: effectivePadding, child: current);
398      }
399
400      if (color != null) {
401        current = ColoredBox(color: color!, child: current);
402      }
403
```

```
246   class Container extends StatelessWidget {
382     Widget build(BuildContext context) {
418     }
419
420     if (foregroundDecoration ≠ null) {
421       current = DecoratedBox(
422         decoration: foregroundDecoration!,
423         position: DecorationPosition.foreground,
424         child: current,
425       );
426     }
427
428     if (constraints ≠ null) {
429       current = ConstrainedBox(constraints: constraints!, child: current);
430     }
431
432     if (margin ≠ null) {
433       current = Padding(padding: margin!, child: current);
434     }
435
436     if (transform ≠ null) {
437       current = Transform(transform: transform!, alignment: transformAlignment, child: current);
438     }
439
440     return current!;
441   }
```

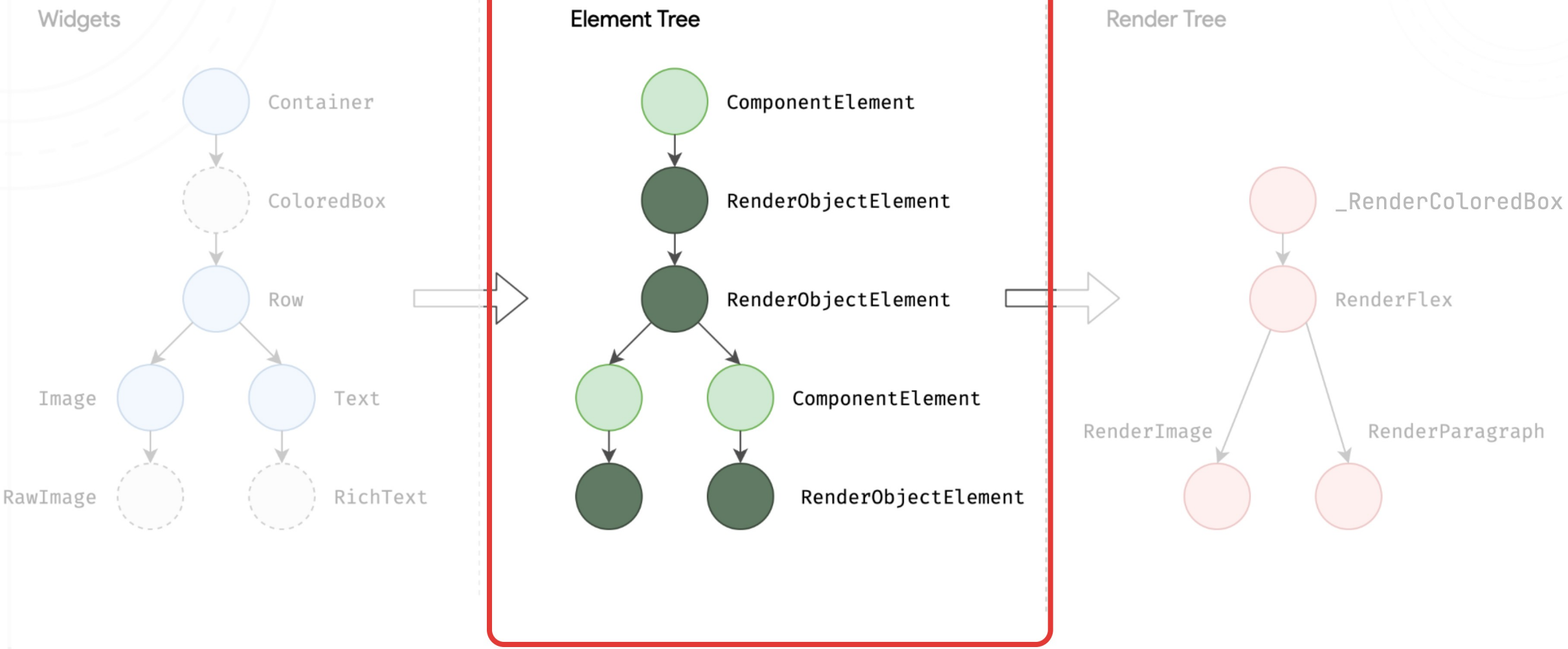
We build digital products.

LeanCode

lekkie i niezmienne

trwałe i mutowalne

layout · paint · dostępność



Źródło: https://docs.flutter.dev/resources/architectural-overview#layout-and-rendering

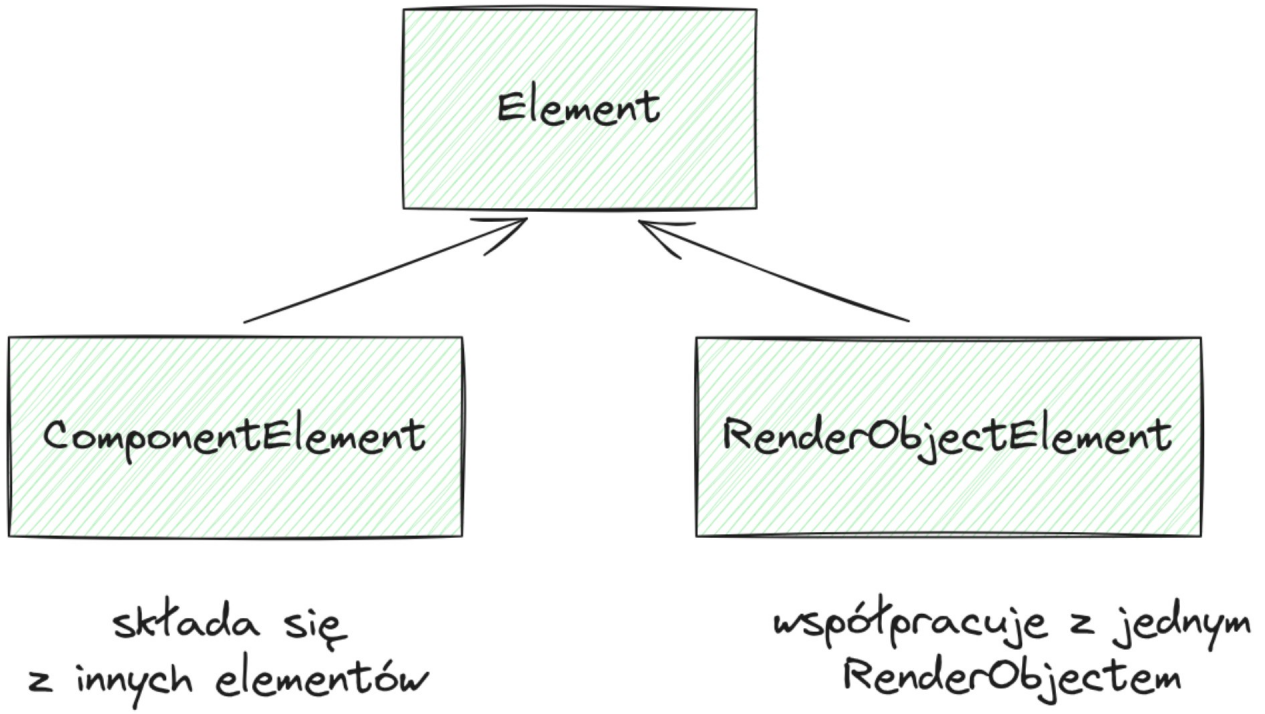
We build digital products.

LeanCode

Elementy

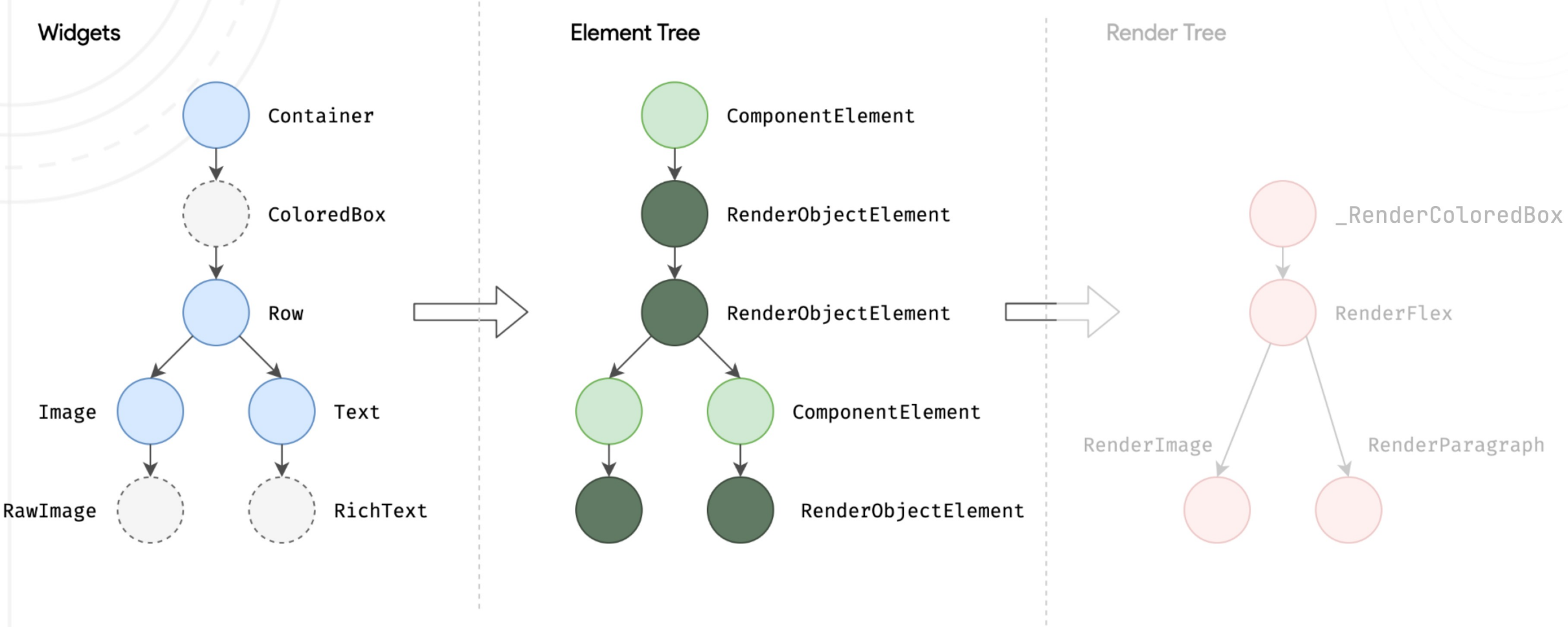
❝ Instancja widgetu w konkretnym miejscu w drzewie.

Element class - widgets library - Dart API



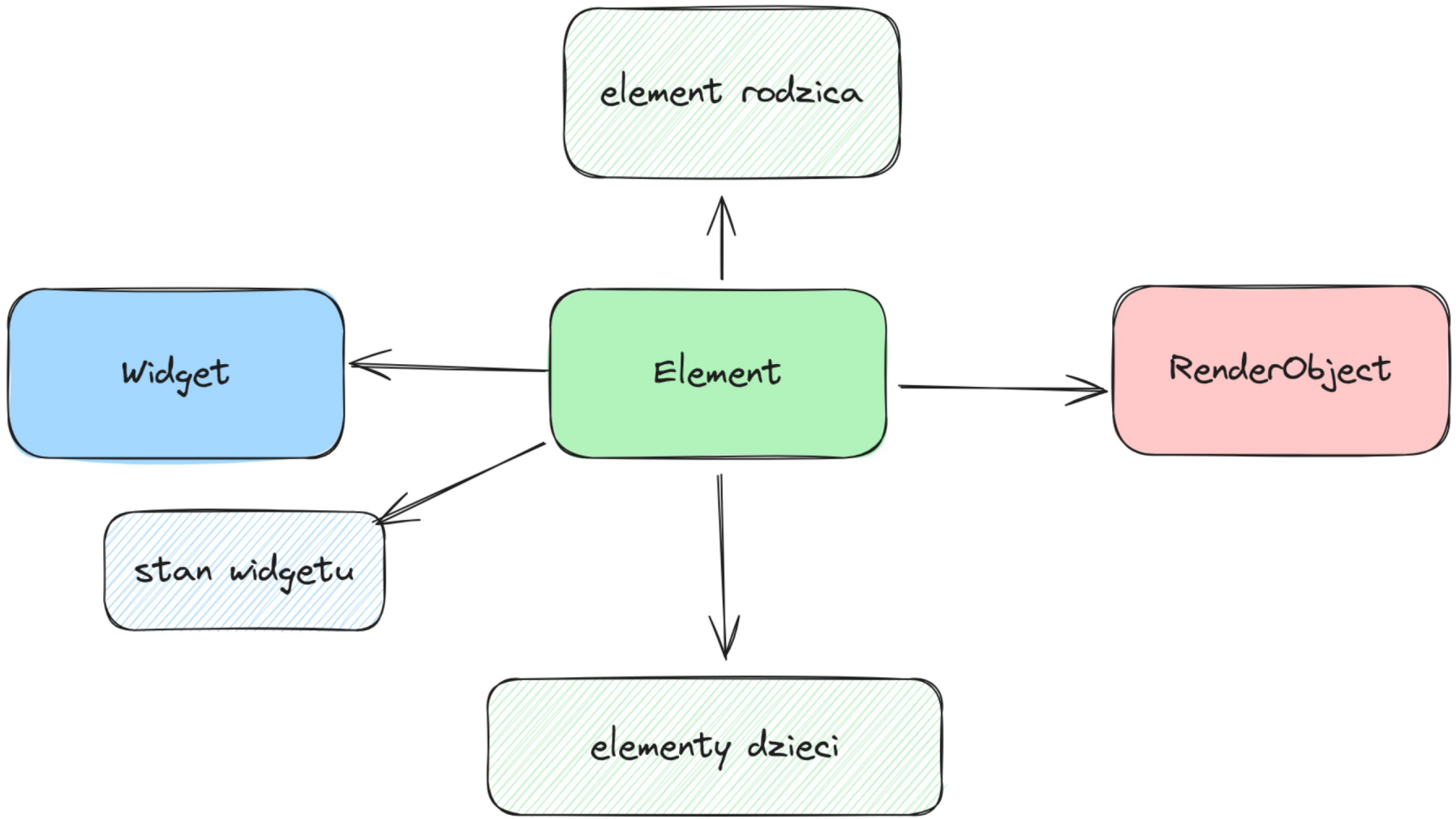
We build digital products.
LeanCode

lekkie i niezmienne | **trwałe i mutowalne** | layout · paint · dostępność



Źródło: https://docs.flutter.dev/resources/architectural-overview#layout-and-rendering

We build digital products.
LeanCode



We build digital products.
LeanCode

Budowanie vs tworzenie widgetów

Co się dzieje, gdy wywołasz

```
final widget = MyWidget();
```

We build digital products.

LeanCode

Konstruktor
po prostu tworzy obiekt widgetu

```dart
class MyWidget extends StatelessWidget {
  const MyWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.amber.shade100,
      width: 100,
      height: 100,
      child: Row(
        children: [
          Image.network('https://picsum.photos/200/300'),
          Text('Hello World!'),
        ],
      ),
    );
  }
}
```

We build digital products.

LeanCode

Konstruktor
po prostu tworzy obiekt widgetu

hierarchia klas

konstruktor każdej z nich – i to cała jego treść

```
const MyWidget({super.key});

const StatelessWidget({super.key});

const Widget({this.key});

const DiagnosticableTree();
```

Żaden konstruktor w całym łańcuchu nie ma logiki – tylko przekazuje key w górę, a Widget go zapisuje.

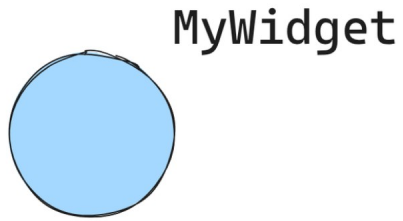
We build digital products.
LeanCode

```dart
final widget = MyWidget();
```

- ! build się nie odpala
- ! Flutter w tym nie uczestniczy
- ! Powstaje po prostu lekki obiekt i tyle

Budowanie drzewa widgetów

```
runApp(MyWidget())
```




```
efekty:

a) powstaje instancja widgetu MyWidget
b) framework rozpoczyna inicjalizację
c) MyWidget trafia do drzewa
   (jako dziecko View tworzonego przez runApp)
d) MyWidget musi zostać zbudowany
```

pętla z początku wykładu

1. User input
2. Animation
3. **Build**
4. Layout
5. Paint
6. Composition
7. Rasterize

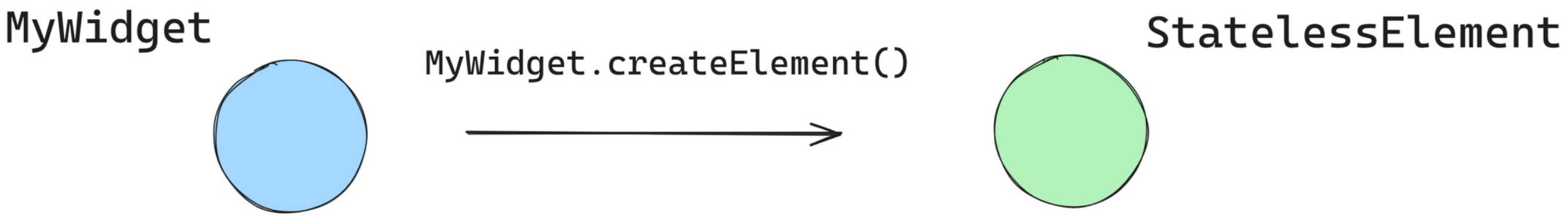
↻ i tak co klatkę (vsync)

runApp planuje pierwszą klatkę – pętla rusza, a w kroku **Build** MyWidget zostaje zbudowany

We build digital products

LeanCode

Budowanie drzewa widgetów

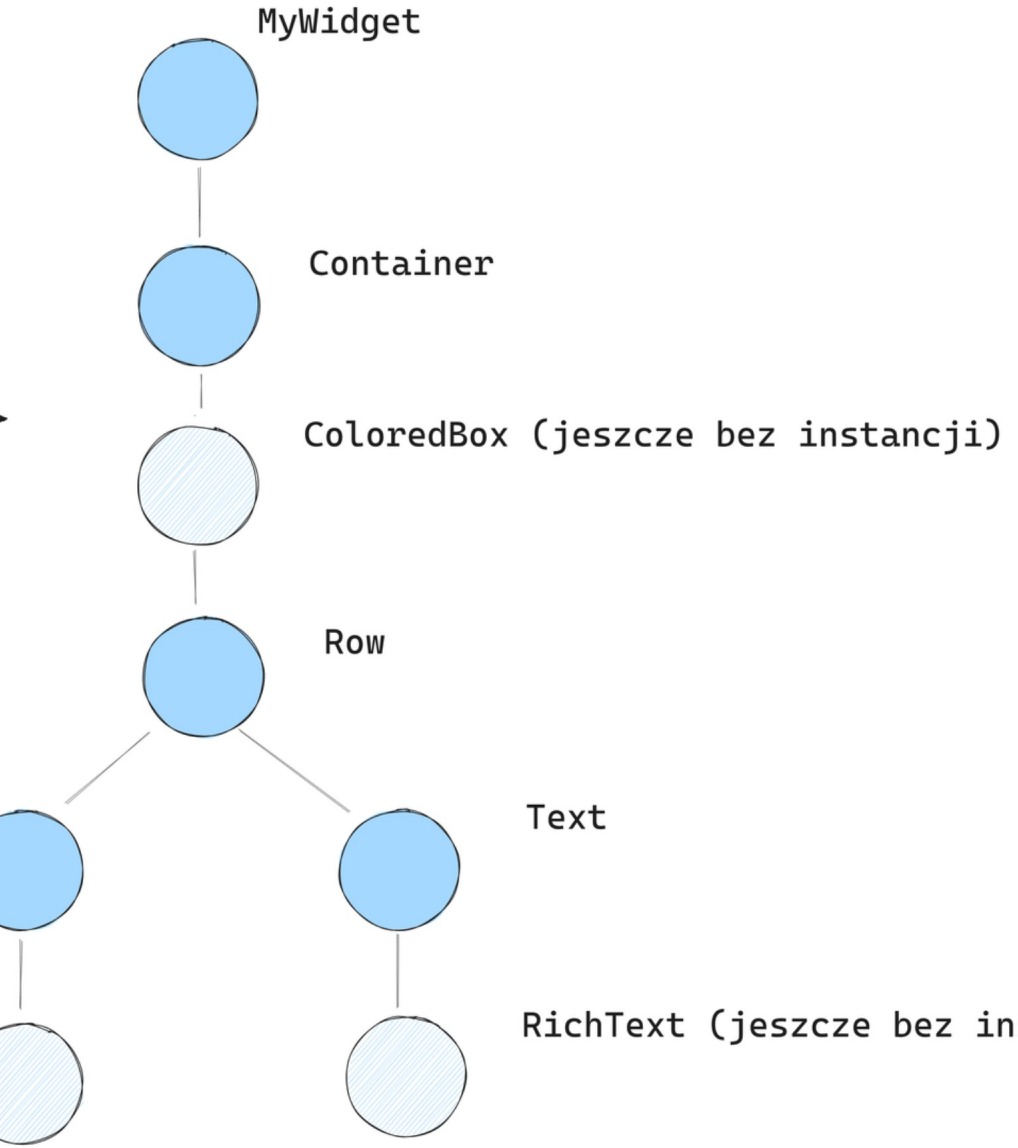


We build digital products.

LeanCode

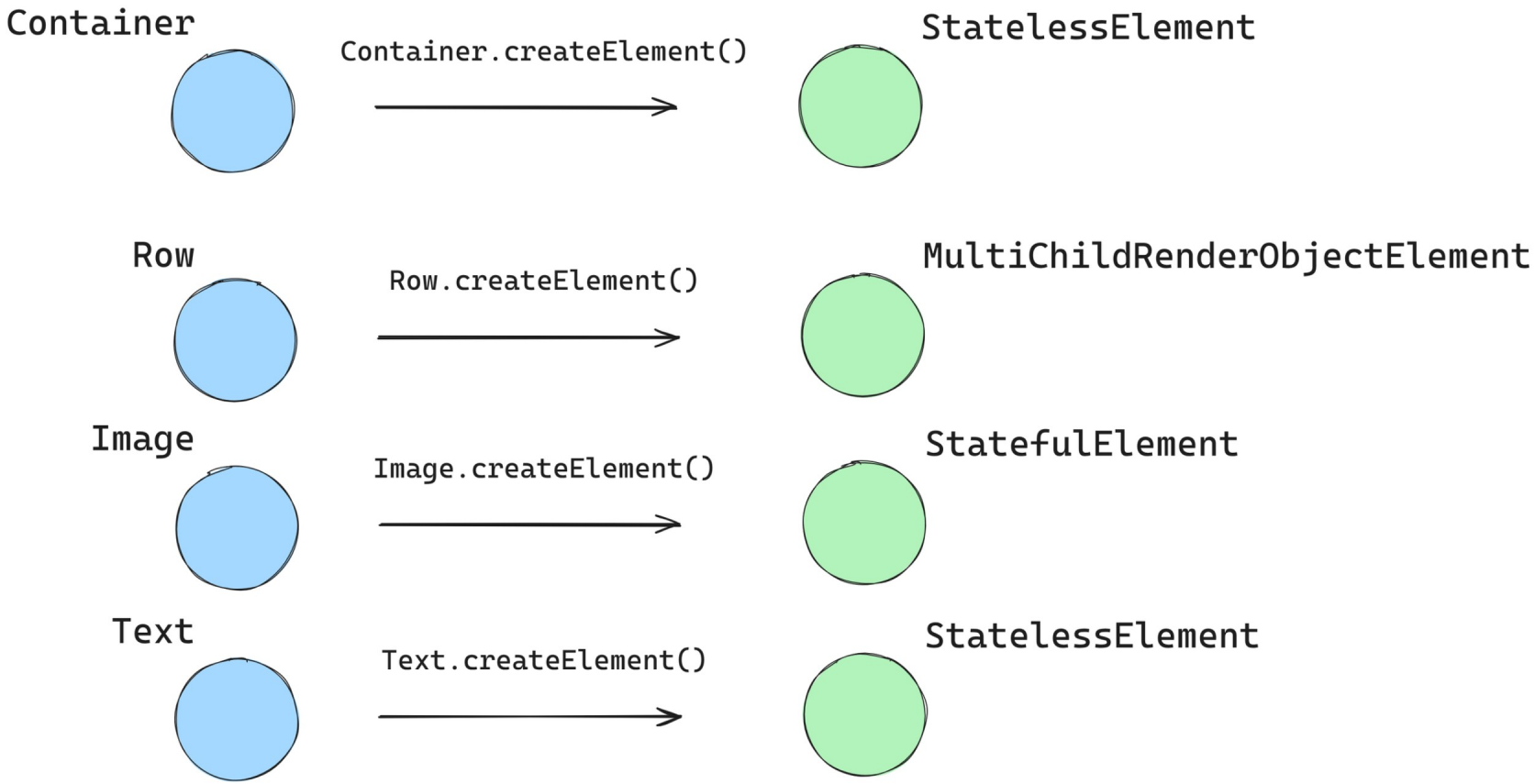
Budowanie drzewa widgetów

MyWidget.build(context)



efekty metody build:
a) powstają instancje Container, Row, Image i Text
b) build zwraca Container – opis poddrzewa
c) framework tworzy dla niego element
   i buduje go w kolejnym kroku

Budowanie drzewa widgetów

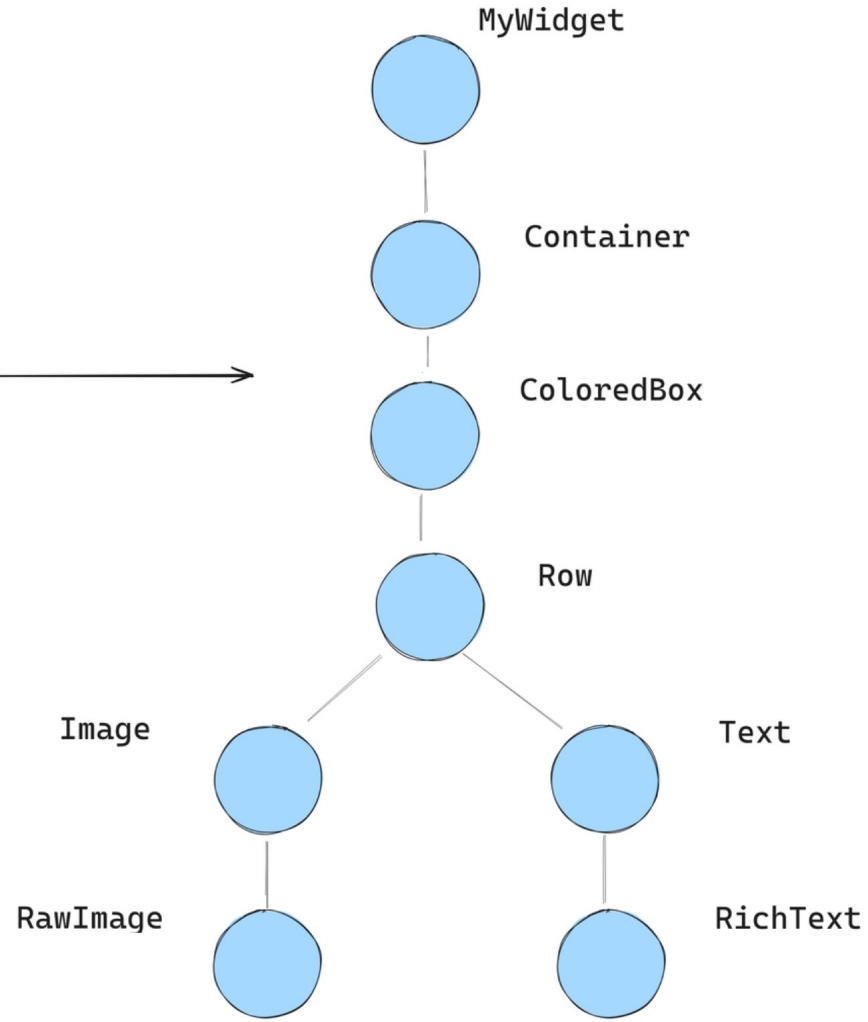


We build digital products.

LeanCode

Budowanie drzewa widgetów

```
Container.build(context)

    Row - bez build()

_ImageState.build(context)

    Text.build(context)
```




```
Row to RenderObjectWidget -
zamiast build() tworzy RenderObject.
Image jest stanowy -
build() ma jego State.
```

We build digital products.
LeanCode

Przebudowa drzewa

Drzewo już stoi. Co się dzieje przy kolejnym buildzie?

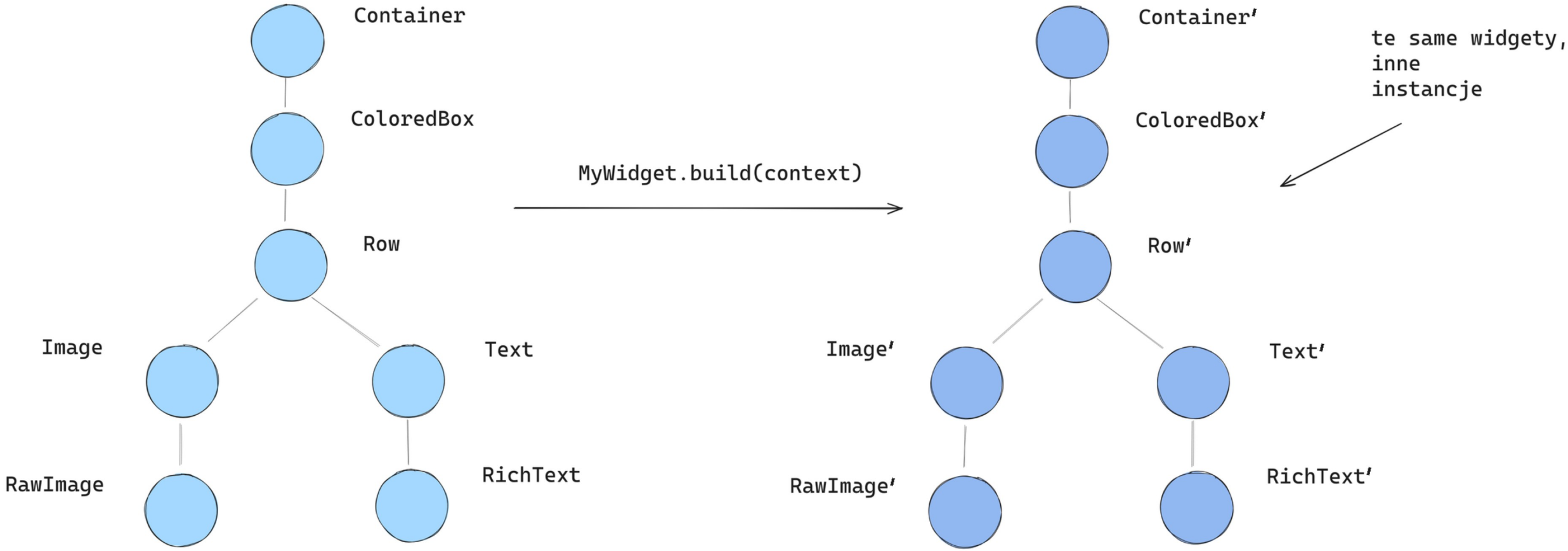
We build digital products.

LeanCode

```dart
class MyWidget extends StatelessWidget {
  const MyWidget({super.key});

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.amber.shade100,
      width: 100,
      height: 100,
      child: Row(
        children: [
          Image.network('https://picsum.photos/200/300'),
          Text('Hello World!'),
        ],
      ),
    );
  }
}
```

We build digital products.

LeanCode



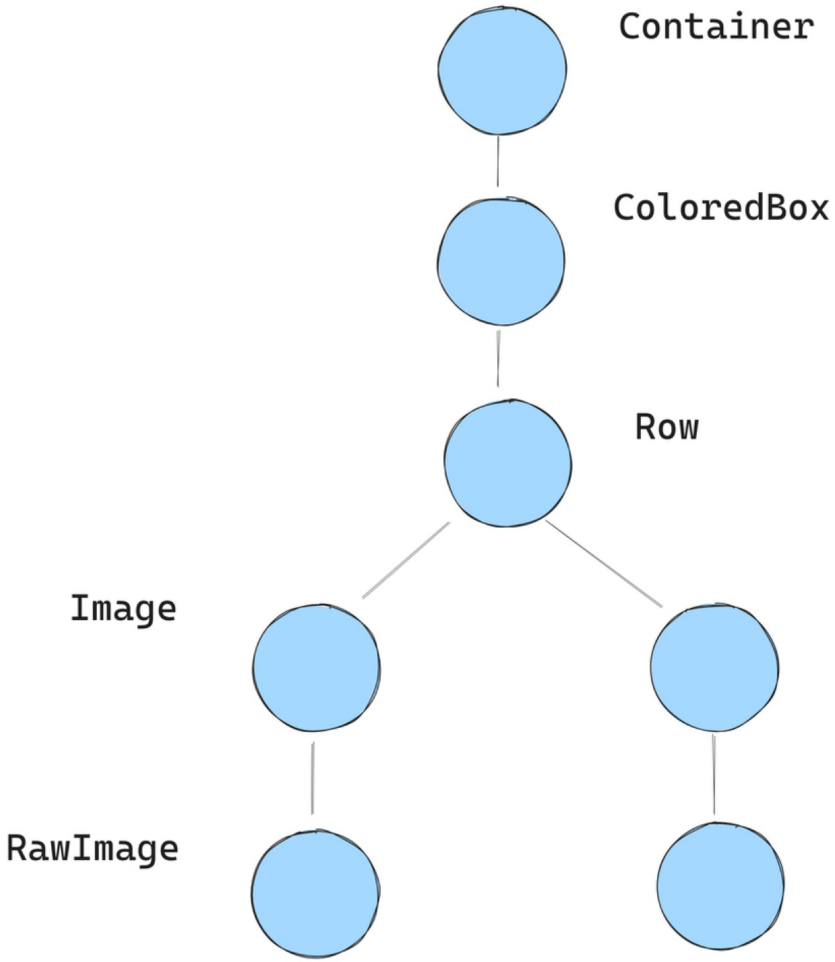
We build digi

LeanCode

```dart
class MyWidget2 extends StatelessWidget {
  const MyWidget2({super.key, required this.showImage});

  final bool showImage;

  @override
  Widget build(BuildContext context) {
    return Container(
      color: Colors.amber.shade100,
      width: 100,
      height: 100,
      child: Row(
        children: [
          if (showImage) Image.network('https://picsum.photos/200/300'),
          Text('Hello World!'),
        ],
      ),
    );
  }
}
```

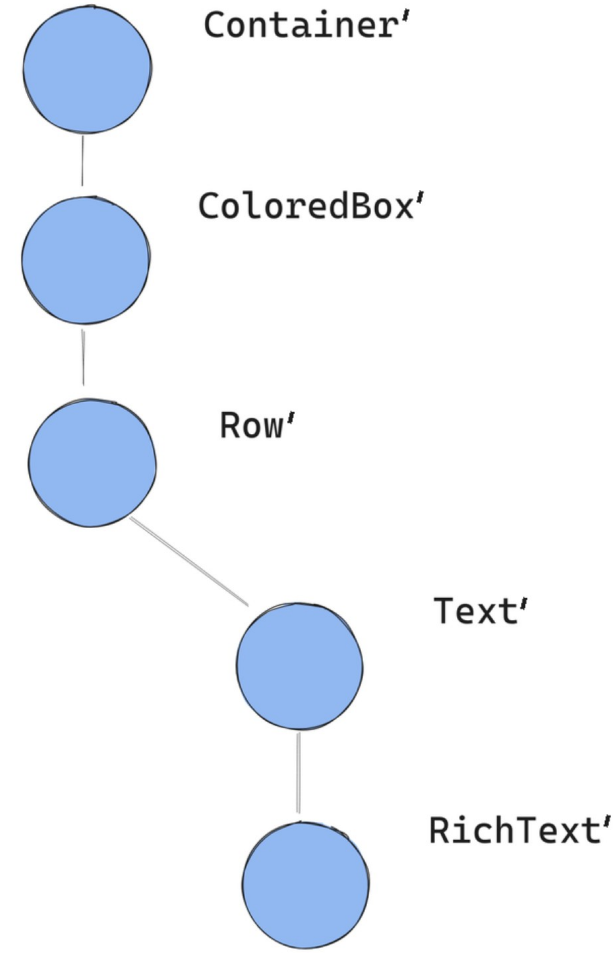
We build digital products.

MyWidget2(showImage: true)



MyWidget2.build(context)

MyWidget2(showImage: false)



We build digital products.

LeanCode

A co z Elementami?

We build digital products.

LeanCode

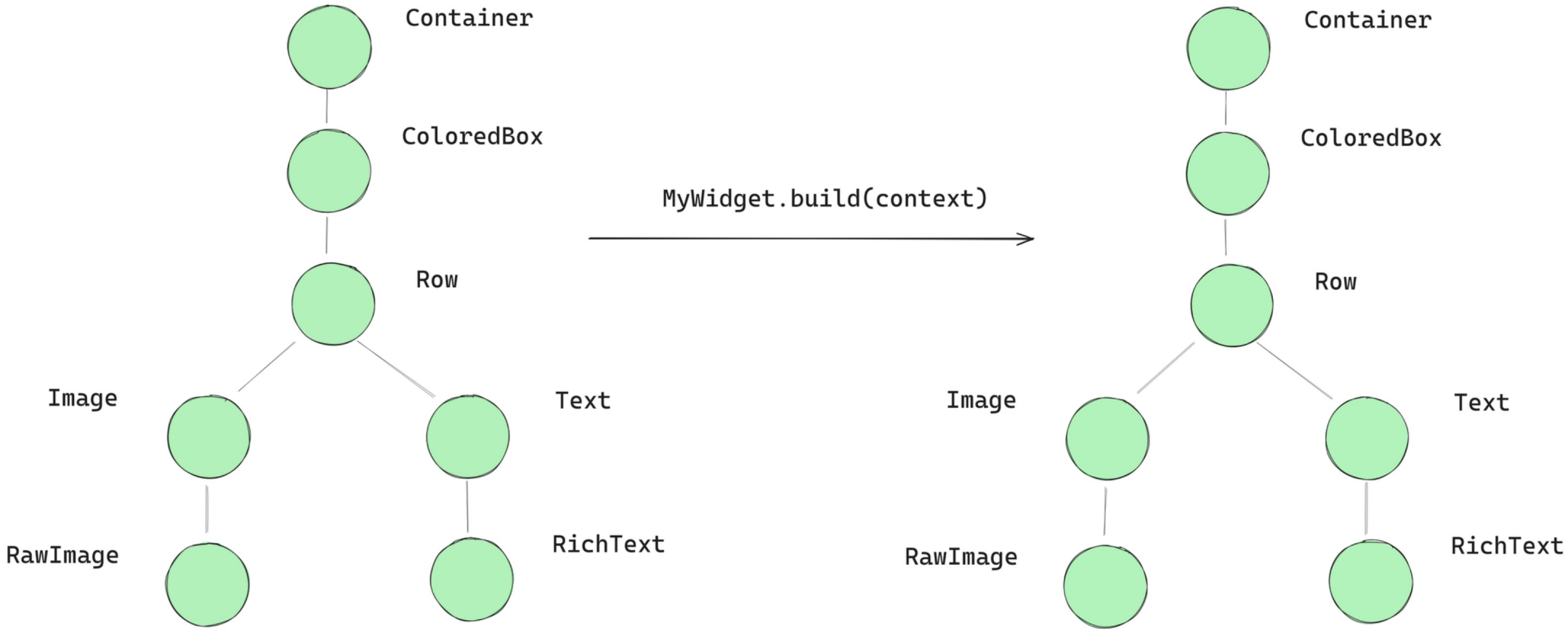
Flutter sprawdza widget na tej samej pozycji w drzewie

1. identical(A, B)? → nic do zrobienia
2. ten sam runtimeType i key? → element.update(B)
3. w przeciwnym razie → nowy element

We build digital products.
LeanCode

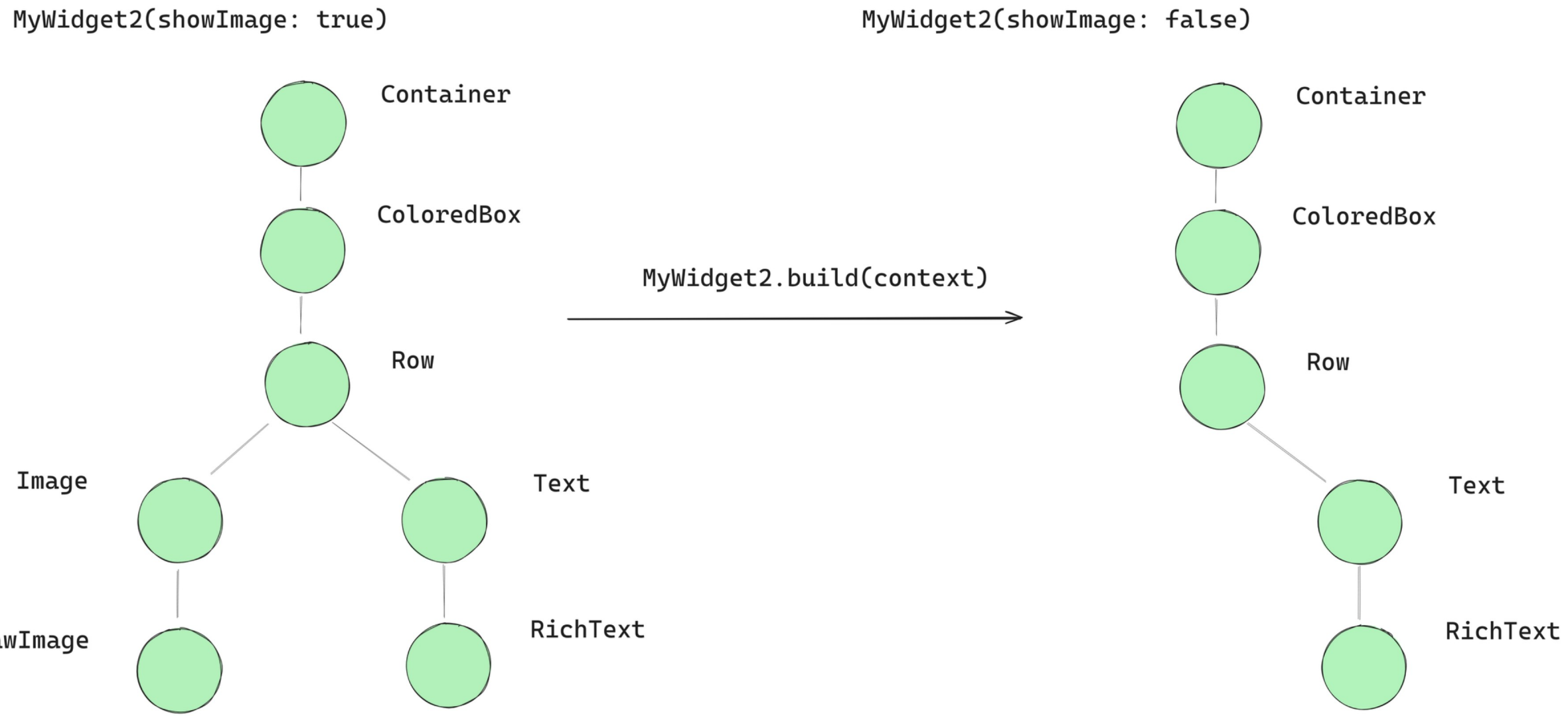
Elementy bez zmian

Ten sam typ i klucz → **te same obiekty Element**, dostają tylko update(newWidget)



Elementy zostają – wylatują tylko Image i RawImage

A co, gdy usunięty widget ma ten sam typ co sąsiad? → wrócimy do tego przy kluczach



We build digital products.
LeanCode

Elementy są mutowalne i trwałe

```dart
abstract class Element implements BuildContext {
  Widget get widget;              // changes over time!

  // lifecycle
  void mount(Element? parent, Object? newSlot);
  void activate(); void deactivate(); void unmount();

  // got a new widget? update in place
  void update(covariant Widget newWidget);
  void markNeedsBuild();

  // managing children
  Element? updateChild(Element? child, Widget? newWidget, Object? newSlot);
  List<Element> updateChildren(List<Element> oldChildren, List<Widget> newWidgets);
  Element inflateWidget(Widget newWidget, Object? newSlot);
  void deactivateChild(Element child);
  void forgetChild(Element child);
  ...
}
```

Widget

@immutable, same final pola. Tani, przy każdym build powstaje nowy

Element

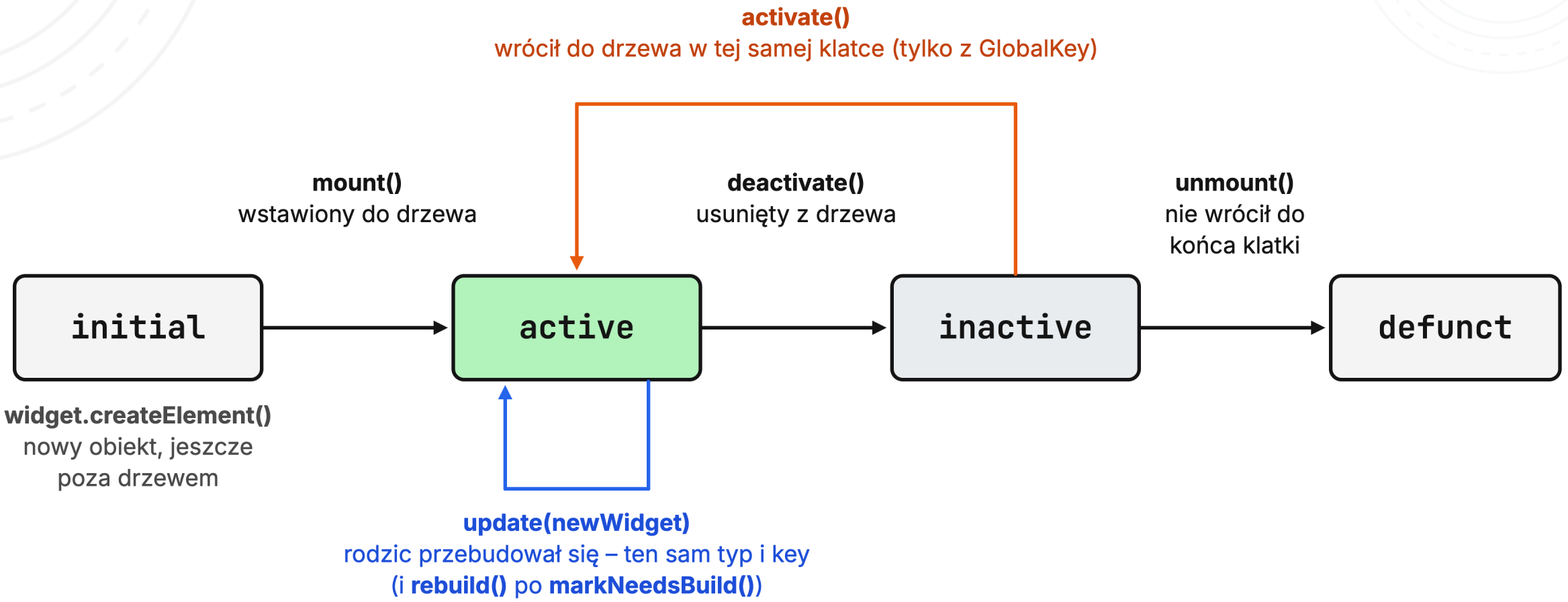
mutowalny i długowieczny. Żyje między buildami, dostaje nowe widgety i zmienia się w miejscu

Pełne API: Element class – Flutter API · sygnatury uproszczone

We build digital products.

LeanCode

Cykl życia elementu



Nazwy stanów jak w kodzie Fluttera: _ElementLifecycle.initial / active / inactive / defunct

We build digital products.
LeanCode

To czym jest BuildContext?

We build digital products.

LeanCode

Element class - widgets library - Dart API

BuildContext class - widgets library - Dart API

Element class

Inheritance

Object › DiagnosticableTree › Element

Implemented types

BuildContext

BuildContext objects are actually Element objects. The BuildContext interface is used to discourage direct manipulation of Element objects.

Implementers

Element

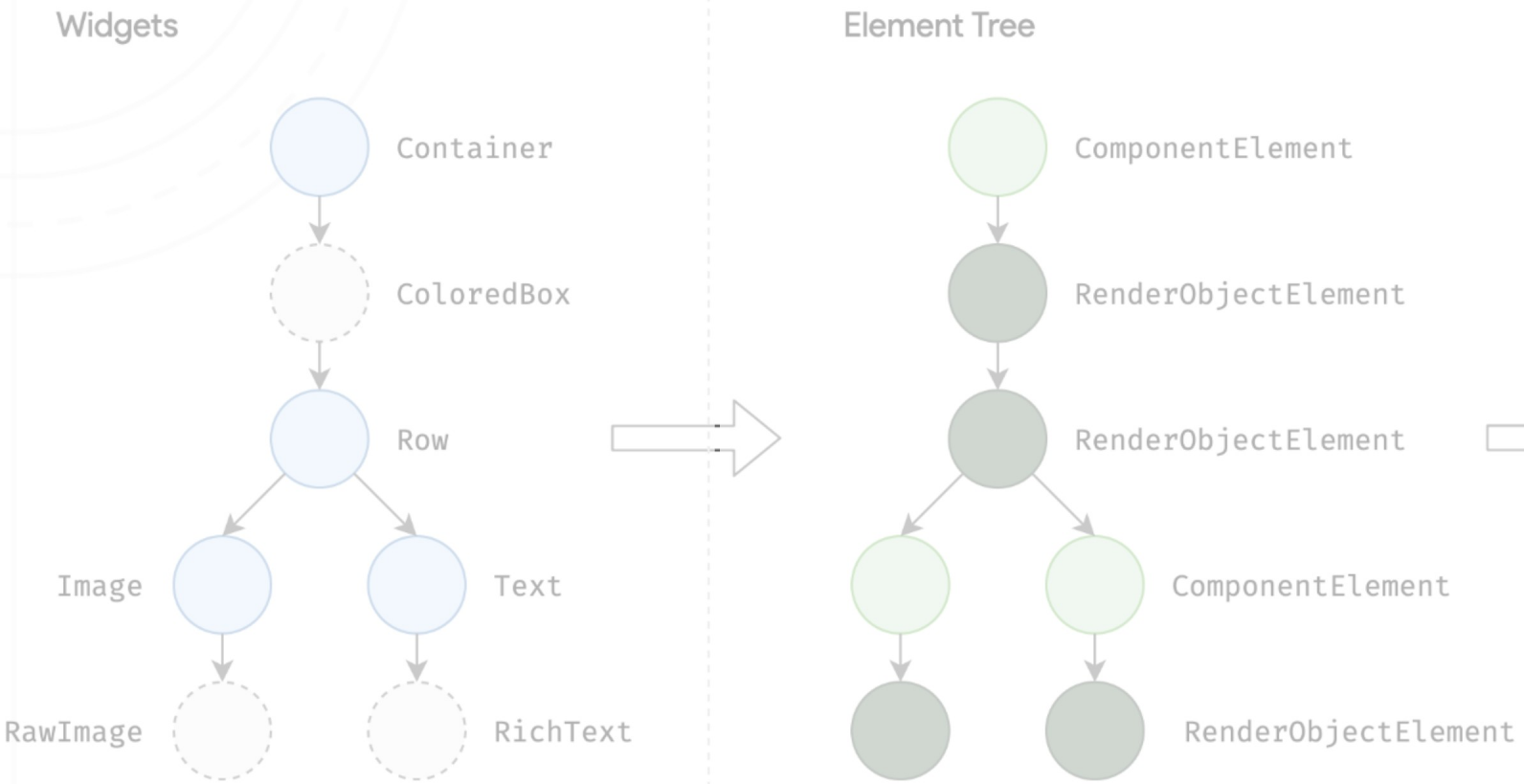
We build digital products.

LeanCode

lekkie i niezmienne

trwałe i mutowalne

layout · paint · dostępność



Źródło: https://docs.flutter.dev/resources/architectural-overview#layout-and-rendering

RenderObjecty

```
/// Compute the layout for this render object.
/// [...]
@protected
void performLayout();

/// Paint this render object into the given context at the given offset.
/// [...]
void paint(PaintingContext context, Offset offset);

/// Report the semantics of this node, for example for accessibility purposes.
/// [...]
@protected
void describeSemanticsConfiguration(SemanticsConfiguration config);
```

We build digital products.

LeanCode

RenderObjecty

- ! To już niższy poziom
- ! Na co dzień raczej ich nie dotykasz
- ! Bierz gotowe widgety, chyba że naprawdę potrzebujesz własnego layoutu[1]

[1] Własny layout bez pisania RenderObjectu: boxy – pub.dev

We build digital products.

LeanCode

A co z dotykiem?

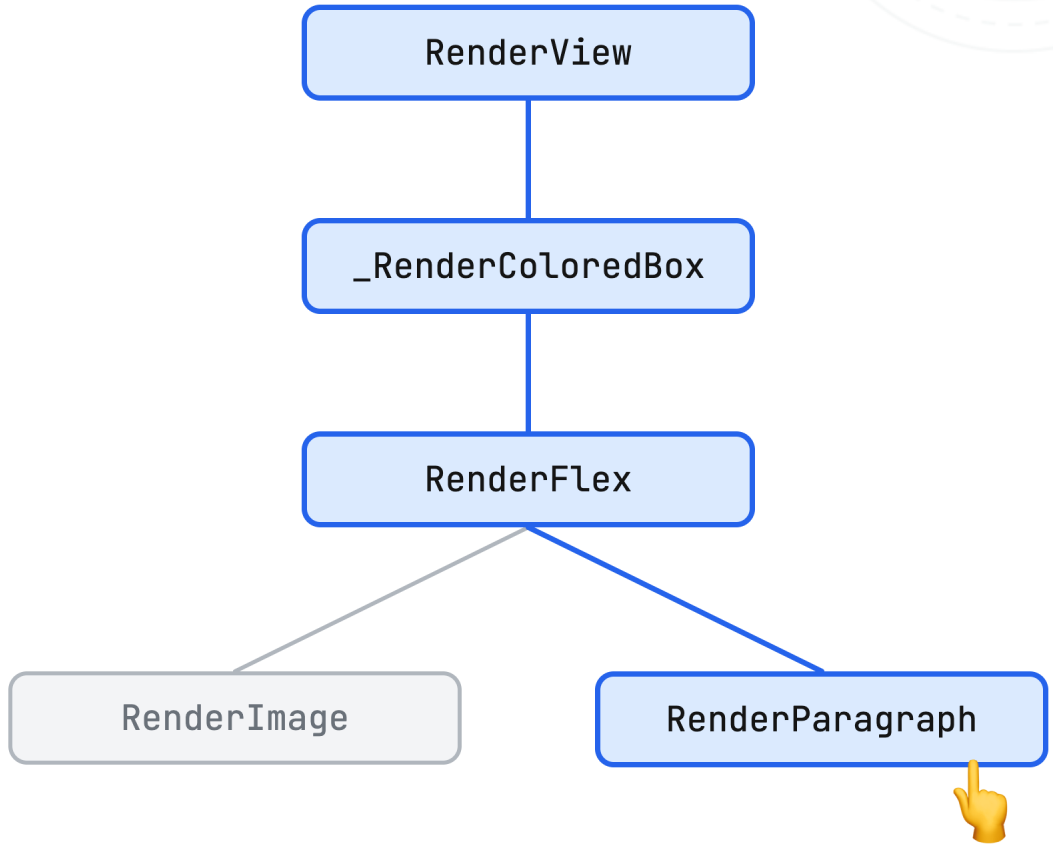
Hit testing, gesty i callbacki

We build digital products.

LeanCode

Hit testing – kto dostał tapnięcie?

1. Palec dotyka ekranu → leci **PointerDownEvent** z pozycją (x, y)
2. Flutter woła **hitTest** od korzenia drzewa render objectów
3. Każdy RenderBox pyta: „czy ten punkt jest w moich granicach?" Jeśli tak – sprawdza dzieci (zaczynając od tego na wierzchu)
4. Wynik to lista trafionych obiektów: od najgłębszego aż do korzenia
5. Zdarzenie – i kolejne z tego dotyku (move, up) – trafia do każdego z nich




```
HitTestResult: [RenderParagraph, RenderFlex,
_RenderColoredBox, RenderView]
```

We build digital products.
LeanCode

GestureDetector

- to zwykły widget – pod spodem ma render object, który bierze udział w hit testingu
- z surowych zdarzeń robi gesty: tap, double tap, long press, drag, scale...
- kilka gestów naraz? Wygrywa jeden – decyduje **gesture arena**[1] (np. tap vs scroll w ListView)
- **behavior: opaque** łapie tapnięcia też w „pustych" miejscach
- zwykły przycisk? Bierz InkWell albo gotowy Button – masz ripple i dostępność za darmo

```dart
GestureDetector(
  behavior: HitTestBehavior.opaque,
  onTap: () => print('tap'),
  onDoubleTap: () => print('double tap'),
  onLongPress: () => print('long press'),
  onHorizontalDragUpdate: (details) {
    print(details.delta.dx);
  },
  child: const Card(
    child: Text('Tap me'),
  ),
)
```

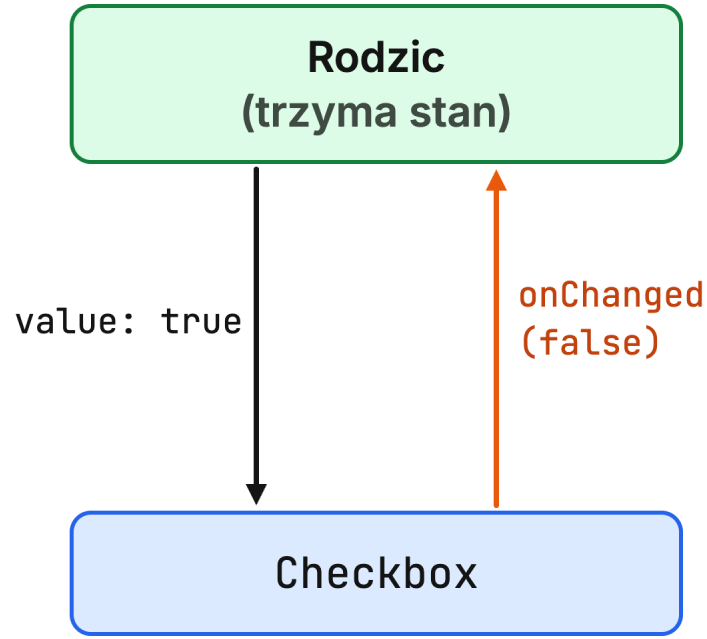
[1] Gesture disambiguation – docs.flutter.dev

We build digital products.

LeanCode

Konwencja onCośtam

- nazwa = **on** + zdarzenie: onTap, onPressed, onChanged, onSelected, onSubmitted
- typ to po prostu funkcja: **VoidCallback** (bez danych) albo **ValueChanged<T>** (z nową wartością)
- callback jest nullable – **null** zwykle znaczy „wyłączone” (ElevatedButton(onPressed: null) robi się szary)
- widget nie zmienia stanu sam – tylko mówi, co się stało. Decyzja należy do rodzica
- **dane płyną w dół, zdarzenia w górę**



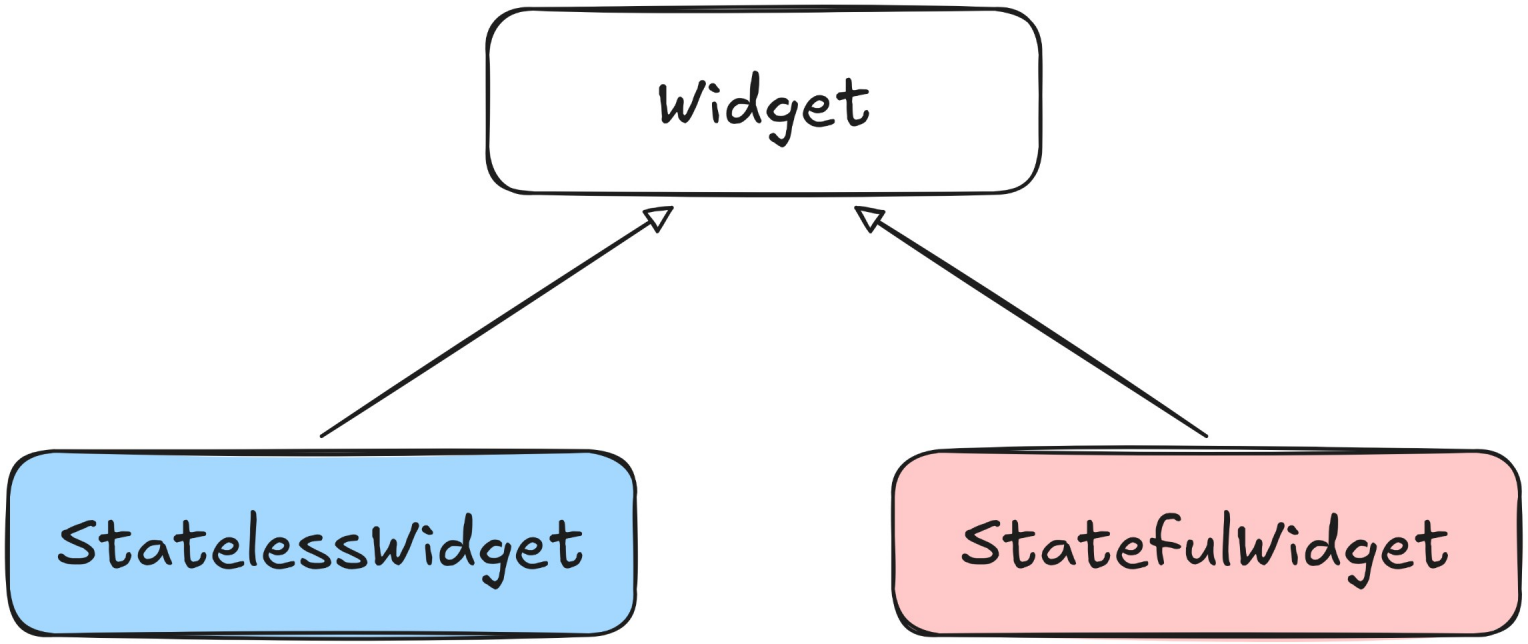
We build digital products.
LeanCode

StatefulWidget

Stan lokalny – reagowanie na input i zmiany w czasie

We build digital products.

LeanCode



We build digital products.
LeanCode

StatelessWidget vs StatefulWidget + State

StatelessWidget lekki i niezmienny

```dart
class Greeting extends StatelessWidget {
  const Greeting({super.key, required this.name});
  final String name; // immutable

  @override
  Widget build(BuildContext context) =>
      Text('Hej $name');
}
```

StatefulWidget lekki i niezmienny

```dart
class Counter extends StatefulWidget {
  const Counter({super.key, required this.step});
  final int step; // immutable

  @override
  State<Counter> createState() => _CounterState();
}
```

State trwały i mutowalny

```dart
class _CounterState extends State<Counter> {
  int _count = 0; // mutable

  @override
  Widget build(BuildContext context) =>
      Text('$_count (+${widget.step})');
  // widget = the current Counter config
}
```

Konfiguracja (pola **final**) siedzi w widgecie – powstaje od nowa przy każdym build.

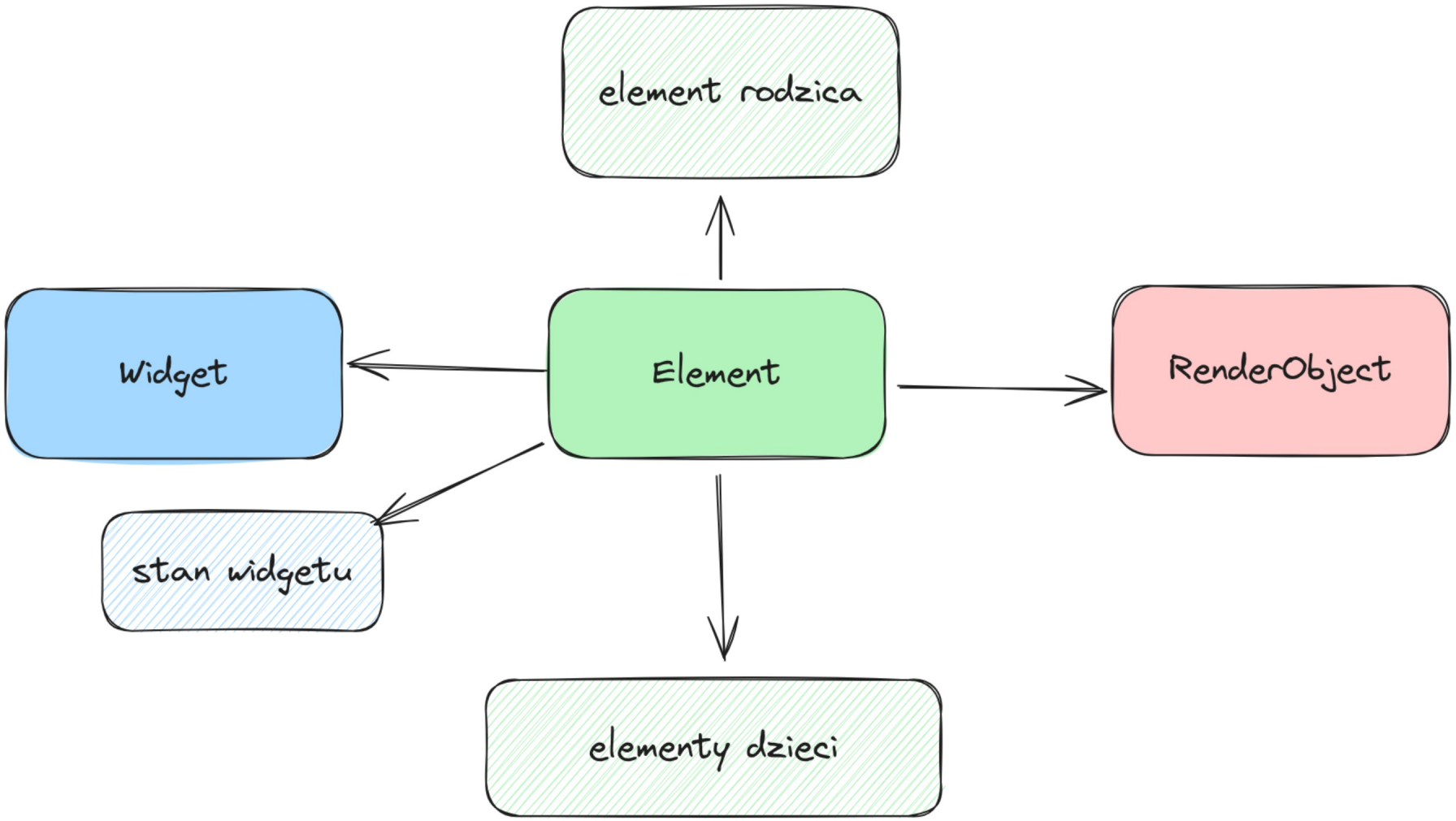
Wszystko, co się **zmienia**, trzymamy w **State** – żyje razem z elementem.

State.widget zawsze wskazuje na aktualną konfigurację.

We build digital products.

LeanCode

📍 Recap



We build digital products.

LeanCode

Robimy widget stanowy

```dart
class StatelessCounter extends StatelessWidget {
  const StatelessCounter({
    super.key,
    required this.value,
    required this.onChanged,
  });

  final int value;
  final ValueChanged<int> onChanged;

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Text(value.toString()),
        ElevatedButton(onPressed: () => onChanged(value + 1), child: Text('+')),
        ElevatedButton(onPressed: () => onChanged(value - 1), child: Text('-')),
      ],
    );
  }
}
```

We build digital products.

LeanCode

```dart
class StatefulCounter extends StatefulWidget {
  const StatefulCounter({
    super.key,
    required this.initialValue,
  });

  final int initialValue;

  @override
  State<StatefulCounter> createState() =>
      _StatefulCounterState();
}
```

```dart
class _StatefulCounterState extends State<StatefulCounter> {
  late int _value;

  @override
  void initState() {
    super.initState();
    _value = widget.initialValue;
  }

  void _increment() {
    setState(() => _value++);
  }

  void _decrement() {
    setState(() => _value--);
  }

  @override
  Widget build(BuildContext context) {
    return Row(
      children: [
        Text(_value.toString()),
        ElevatedButton(onPressed: _increment, child: Text('+')),
        ElevatedButton(onPressed: _decrement, child: Text('-')),
      ],
    );
  }
}
```

We build digital products.

LeanCode

Widgety przychodzą i odchodzą,
State zostaje

We build digital products.

LeanCode

Zmiana stanu

użyj metody setState

```dart
class _CounterState extends State<Counter> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return TextButton(
      onPressed: () {
        setState(() {
          _count++;
        });
      },
      child: Text('Counter: $_count'),
    );
  }
}
```

We build digital products.

LeanCode

Własny widget z callbackiem

```dart
class ColorPicker extends StatelessWidget {
  const ColorPicker({
    super.key,
    required this.selected,
    required this.onSelected,
  });

  final Color selected;
  final ValueChanged<Color>? onSelected; // null = disabled

  @override
  Widget build(BuildContext context) {
    final onSelected = this.onSelected;
    return Row(
      children: [
        for (final color in [Colors.red, Colors.green, Colors.blue])
          GestureDetector(
            onTap: onSelected == null ? null : ()
⇒ onSelected(color),
            child: ColorDot(color, isSelected: color == selected),
          ),
      ],
    );
  }
}
```

```dart
// in the parent's State
ColorPicker(
  selected: _color,
  onSelected: (color) {
    setState(() ⇒ _color = color);
  },
)
```

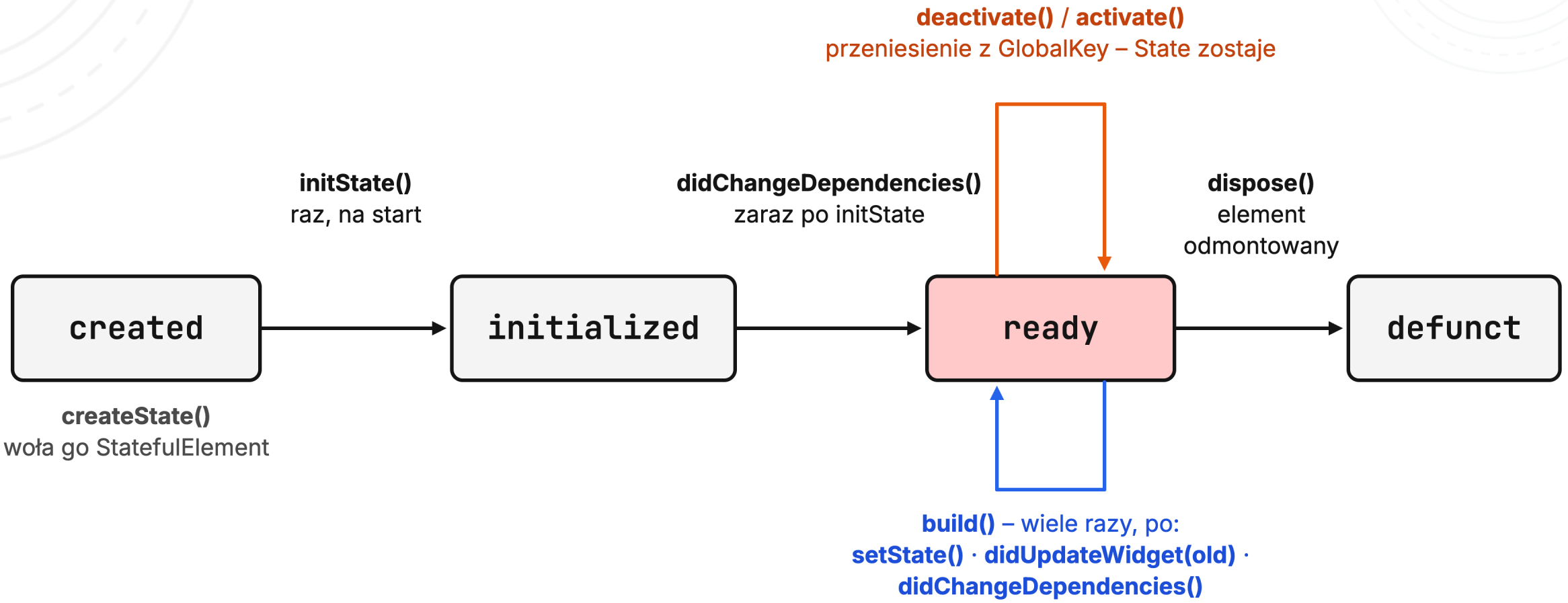
ColorPicker jest **stateless** – stan siedzi u rodzica, a picker tylko mówi, co kliknięto.

Lokalna zmienna onSelected = żeby Dart mógł ją „wypromować" do typu nie-null.

We build digital products.

LeanCode

Cykl życia State



Nazwy stanów jak w kodzie Fluttera: _StateLifecycle.created / initialized / ready / defunct · mounted = true do dispose()

We build digital products.
LeanCode

Element vs State – cykle życia obok siebie

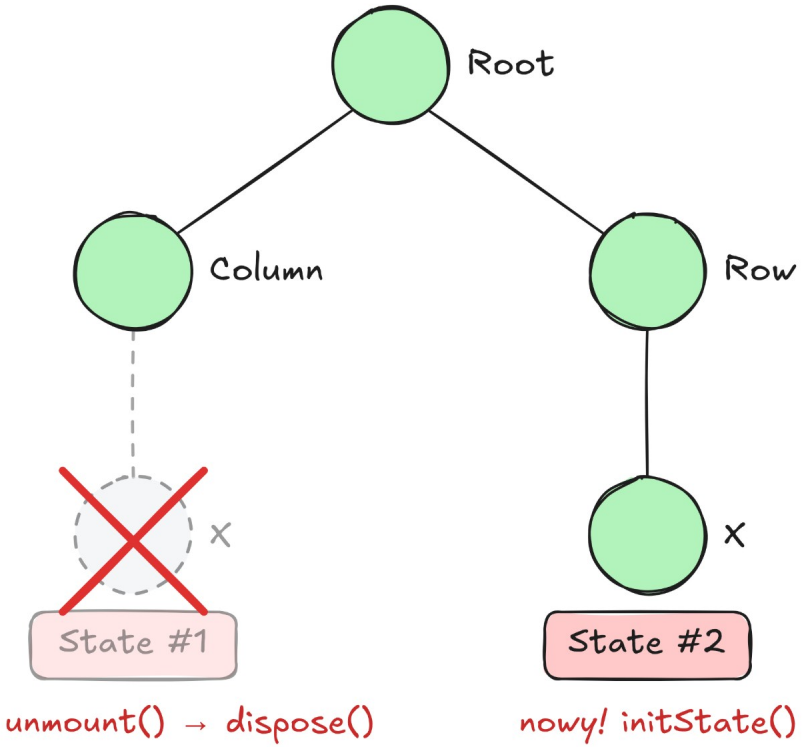
	Element	State
Kto tworzy	`widget.createElement()`	`StatefulWidget.createState()`
Stany	`initial → active ⇄ inactive → defunct`	`created → initialized → ready → defunct`
Wejście do drzewa	`mount()`	`initState() + didChangeDependencies()`
Nowy widget od rodzica	`update(newWidget)`	`didUpdateWidget(oldWidget)`
Zmiana InheritedWidget	`didChangeDependencies()`	`didChangeDependencies()`
Przebudowa	`rebuild() → build()`	`build()`
Wypada / wraca (GlobalKey)	`deactivate() / activate()`	`deactivate() / activate()`
Koniec	`unmount()`	`dispose()`

State żyje dokładnie tyle, co jego **StatefulElement** – to element woła wszystkie te metody. A co, gdy widget trafi pod innego rodzica? →

We build digital products.
LeanCode

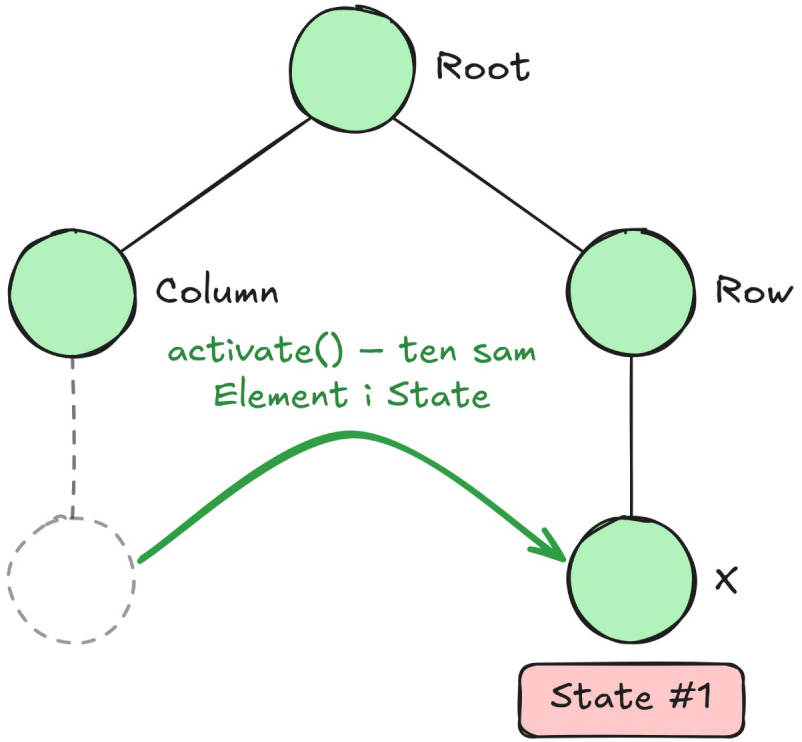
Przeniesienie widgetu pod innego rodzica

bez GlobalKey



stan przepada

z GlobalKey



ten sam Element, State i RenderObject

Warunek: przeniesienie w tej samej klatce – inaczej nieaktywny element zostaje odmontowany · więcej o kluczach za chwilę

We build digital products.
LeanCode

💡 Key takeaway

State jest powiązany z jednym BuildContext (Elementem)

Rodzi się, żyje i umiera razem ze swoim elementem.
Inne miejsce w drzewie = nowy State. Chyba że użyjesz GlobalKey.

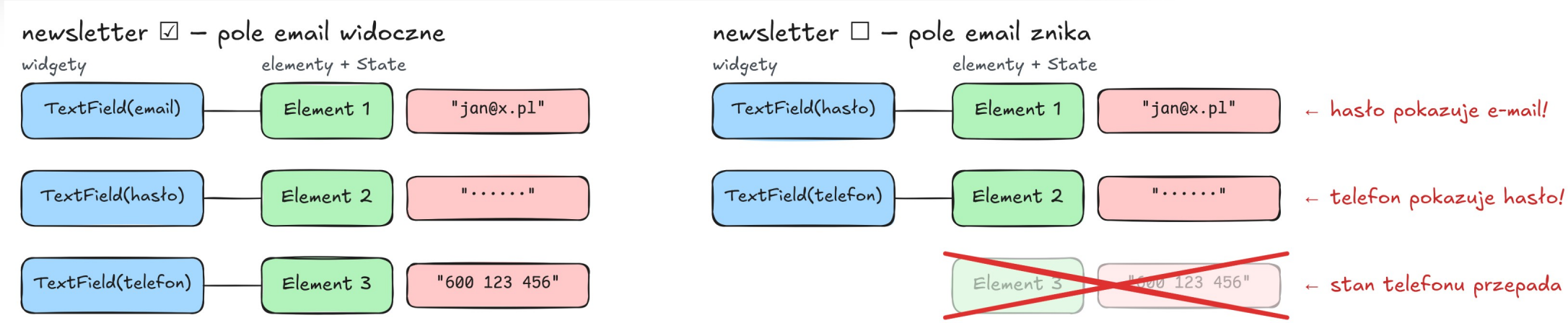
We build digital products.
LeanCode

Problem: chowamy pole w formularzu

```
Column(children: [
  CheckboxListTile(title: Text('Newsletter'), value: _newsletter,
      onChanged: (v) => setState(() => _newsletter = v!)),
  if (_newsletter) TextField(decoration: InputDecoration(labelText: 'email')),
  TextField(decoration: InputDecoration(labelText: 'hasło')),
  TextField(decoration: InputDecoration(labelText: 'telefon')),
]),
```

Pole email pokazuje się tylko przy zaznaczonym newsletterze.

Użytkownik wypełnia formularz i **odznacza checkbox**...



Jak powiedzieć Flutterowi, które pole jest które?

We build digital products.
LeanCode

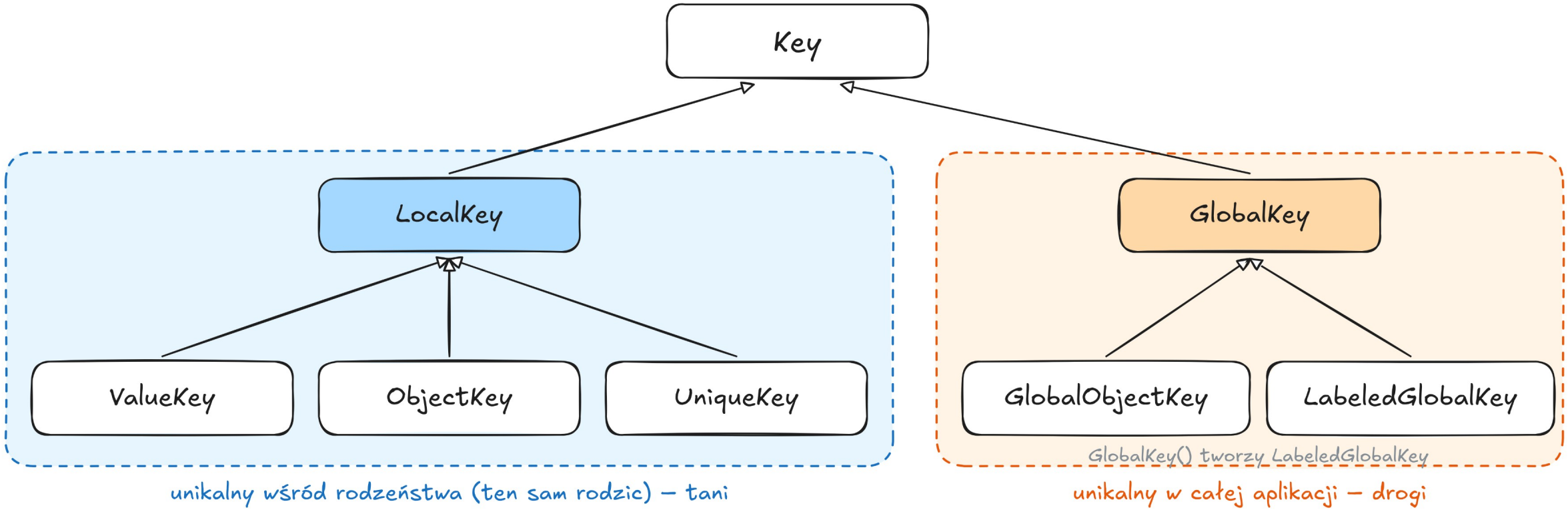
Klucze – identyfikatory widgetów

Dobra dokumentacja, jest też 10-minutowy filmik na YouTube:
Key class - foundation library - Dart API (flutter.dev)

We build digital products.

LeanCode

Rodzaje kluczy



We build digital products.
LeanCode

Kiedy klucze są równe i kiedy ich używać

klucz	równe, gdy...	do czego
`ValueKey(v)` `Key('x') = ValueKey<String>('x')`	v1 == v2	elementy listy z id: ValueKey(todo.id), pola formularza: Key('email') – najczęściej spotykany zapis
`ObjectKey(o)`	identical(o1, o2)	obiekt bez sensownego == i bez id – klucz to sama instancja *(rzadko)*
`UniqueKey()`	tylko ta sama instancja klucza	wymuszenie nowego Elementu i State, np. reset formularza albo restart animacji
`GlobalKey()`	tylko ta sama instancja klucza	Form + currentState!.validate(), przeniesienie widgetu ze stanem, dostęp do contextu / rozmiaru
`GlobalObjectKey(o)`	identical(o1, o2)	GlobalKey wyliczany z obiektu, gdy nie da się przechować instancji klucza *(rzadko)*

identical – ten sam obiekt (równość referencyjna) · == – równość, którą klasa może nadpisać · klucze muszą też mieć ten sam typ

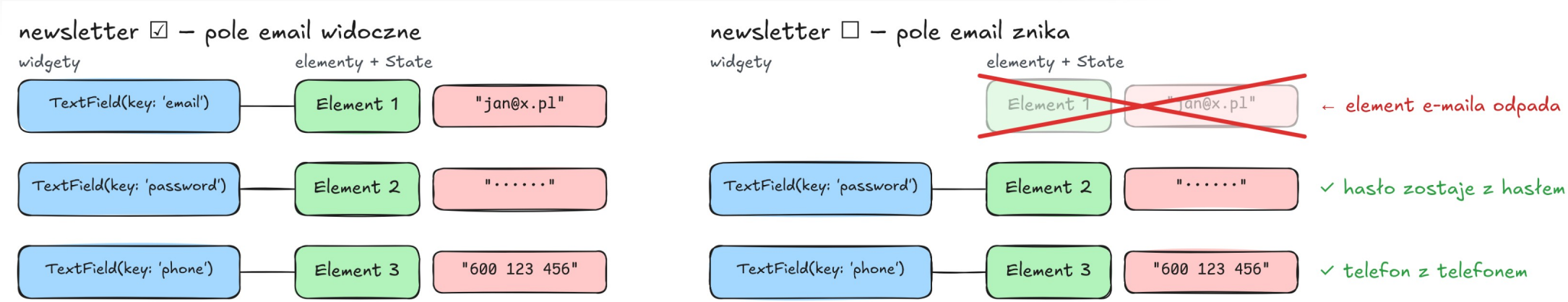
We build digital products.
LeanCode

Rozwiązanie: klucz na każdym polu

```dart
Column(children: [
  CheckboxListTile(/* ... */),
  if (_newsletter)
    TextField(key: const Key('email'), /* ... */),
  TextField(key: const Key('password'), /* ... */),
  TextField(key: const Key('phone'), /* ... */),
]),
```

Teraz Flutter dopasowuje elementy **po kluczu**, a nie po pozycji.

Element e-maila odpada, reszta zostaje ze swoim stanem.



Wystarczy klucz unikalny wśród rodzeństwa – zwykły LocalKey, nie GlobalKey.

We build digital products.
LeanCode

Klucze lokalne: porównywane tylko wśród rodzeństwa

✅ ten sam klucz u różnych rodziców

OK – każdy rodzic porównuje tylko swoje dzieci

```
Column(children: [
  Row(children: [
    Text('A', key: Key('x')),
  ]),
  Row(children: [
    Text('B', key: Key('x')),
  ]),
])
```

❌ ten sam klucz u rodzeństwa

błąd – rodzic nie odróżni dzieci

```
Row(children: [
  Text('A', key: Key('x')),
  Text('B', key: Key('x')),
])

// Duplicate keys found.
```

⚠️ zmiana rodzica

klucz nie pomoże – stan przepada

```
showLeft
  ? Left(child: Field(key: Key('x')))
  : Right(child: Field(key: Key('x')))

// new Element + new State
```

LocalKey (ValueKey, ObjectKey, UniqueKey) działa w obrębie jednej listy children. Przeniesienie między rodzicami – tylko GlobalKey.

We build digital products.

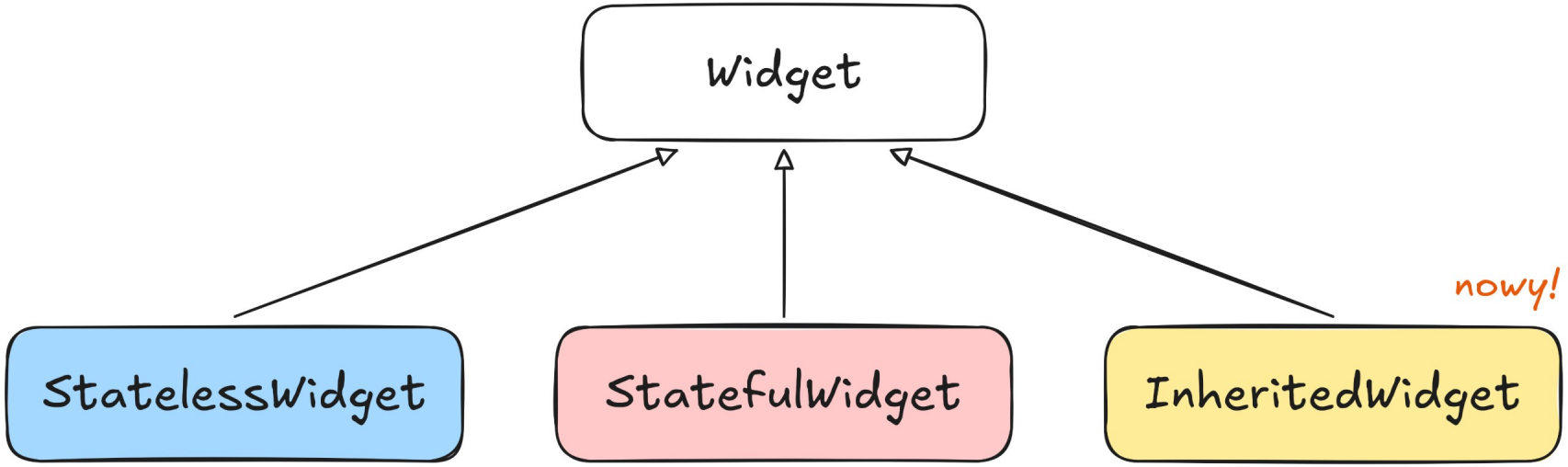
LeanCode

InheritedWidget

przekazywanie danych w tle

We build digital products.

LeanCode



We build digital products.
LeanCode

Po co to komu?

```
Widget build(BuildContext context) {
  return AppRoot(
    userInfo: userInfo,
    child: HomePage(
      userInfo: userInfo,
      child: ProfileCard(userInfo: userInfo),
    ),
  );
}
```

We build digital products.

LeanCode

Po co to komu?

```dart
class NavigatorExample extends StatelessWidget {
  const NavigatorExample({super.key, required this.userInfo});

  final UserInfo userInfo;

  @override
  Widget build(BuildContext context) {
    return Navigator(
      /* ??? How do we pass the userInfo to the next screen? */
    );
  }
}
```

We build digital products.

LeanCode

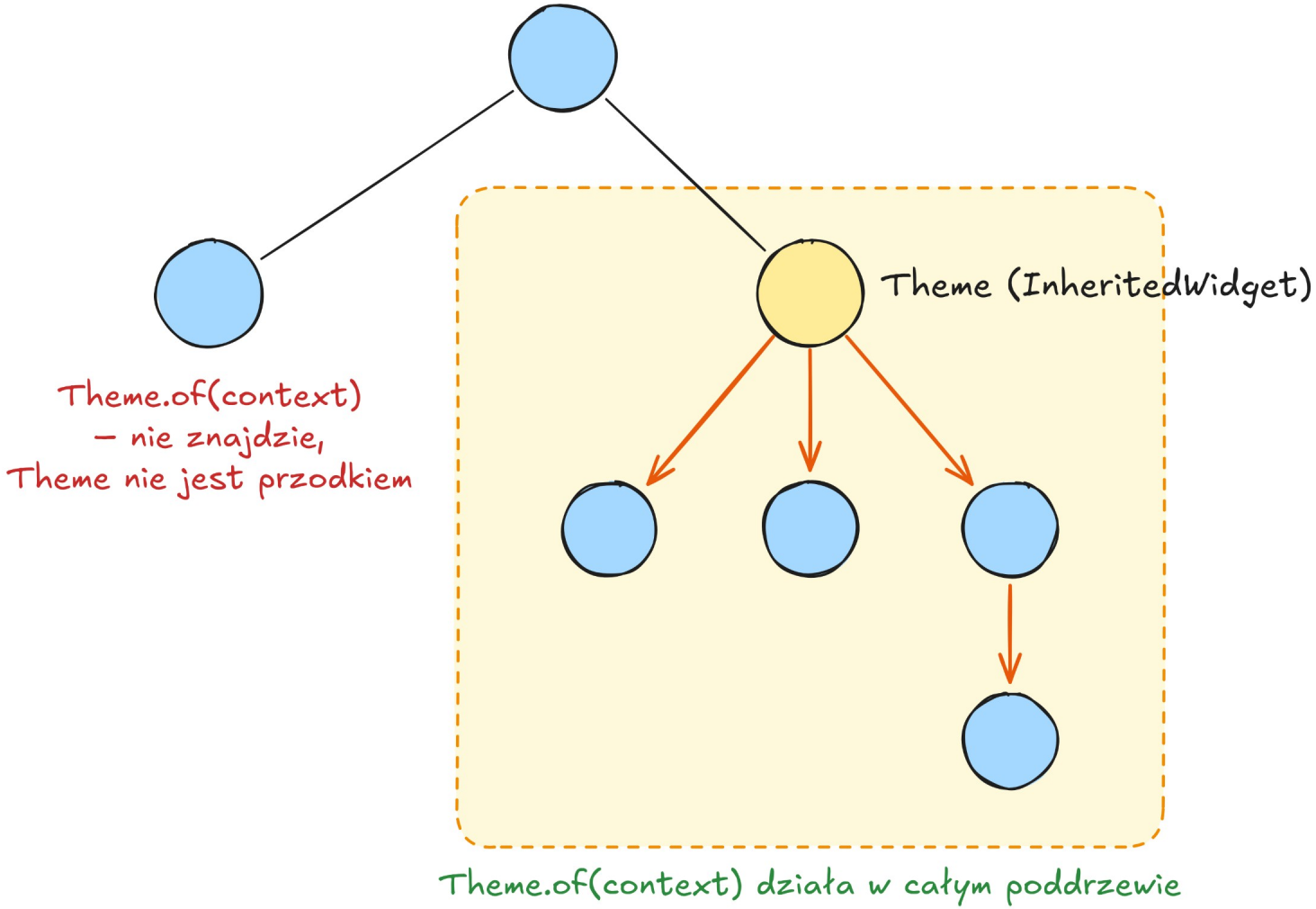
Prop drilling 🤢

```dart
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    final theme = ThemeData.light();

    return MaterialApp(
      theme: theme,
      home: Scaffold(
        backgroundColor: theme.scaffoldBackgroundColor,
        body: Center(child: Text('Hello World!')),
        floatingActionButton: FloatingActionButton(
          onPressed: () {},
          foregroundColor: theme.floatingActionButtonTheme.foregroundColor,
          backgroundColor: theme.floatingActionButtonTheme.backgroundColor,
        ),
      ),
    );
  }
}
```

We build digital products.

LeanCode

Dane płyną w dół



We build digital products.
LeanCode

InheritedWidget

1 Dziedziczymy po InheritedWidget

2 Dorzucamy pola, które chcemy przekazać w dół

3 Dokładamy zwyczajowe helpery of() i maybeOf()
Nie ma providera wyżej w drzewie? of() się wywali, a maybeOf() po prostu zwróci null.

4 Nie powiadamiamy bez potrzeby – szybciej działa

```dart
1 class AppThemeProvider extends InheritedWidget {
    const AppThemeProvider({
      super.key,
2     required this.appTheme,
      required super.child,
    });

2   final AppTheme appTheme;

3   static AppTheme? maybeOf(BuildContext context) {
      return context
          .dependOnInheritedWidgetOfExactType<AppThemeProvider>()
          ?.appTheme;
    }

    static AppTheme of(BuildContext context) {
      final theme = maybeOf(context);
      assert(theme ≠ null, 'No AppThemeProvider found in context');
      return theme!;
    }

4   @override
    bool updateShouldNotify(AppThemeProvider oldWidget) {
      return oldWidget.appTheme ≠ appTheme;
    }
}
```

We build digital products.
LeanCode

InheritedWidget – użycie

1. Wstawiamy wysoko w drzewie

```dart
@override
Widget build(BuildContext context) {
  return AppThemeProvider(
    appTheme: theme,
    child: const SomeComplexHierarchy(),
  );
}
```

AppThemeProvider udostępnia **appTheme** całemu poddrzewu – bez przekazywania przez konstruktory.

AppText pobiera go przez **of(context)** i przebuduje się, gdy motyw się zmieni (updateShouldNotify).

2. Czytamy gdziekolwiek poniżej

```dart
class AppText extends StatelessWidget {
  const AppText(this.text, {super.key});

  final String text;

  @override
  Widget build(BuildContext context) {
    final theme = AppThemeProvider.of(context);

    return Text(
      text,
      style: TextStyle(color: theme.textColor),
    );
  }
}
```

We build digital products.

LeanCode

Provider [pub.dev]

InheritedWidget, tylko łatwiej

We build digital products.

LeanCode

Provider

Nie musisz pisać własnego InheritedWidgeta
→ bierzesz gotowego, generycznego Providera

```dart
@override
Widget build(BuildContext context) {
  return Provider.value(
    value: theme,
    child: const SomeComplexHierarchy(),
  );
}
```

We build digital products.

LeanCode

Provider

```dart
class AppText extends StatelessWidget {
  const AppText(this.text, {super.key});

  final String text;

  @override
  Widget build(BuildContext context) {
    final theme = context.watch<AppTheme>();
    // same as this:
    // final theme = Provider.of<AppTheme>(context);

    return Text(
      text,
      style: TextStyle(color: theme.textColor),
    );
  }
}
```

We build digital products.

LeanCode

Wracamy do BuildContext

of(), zależności i pułapki

We build digital products.

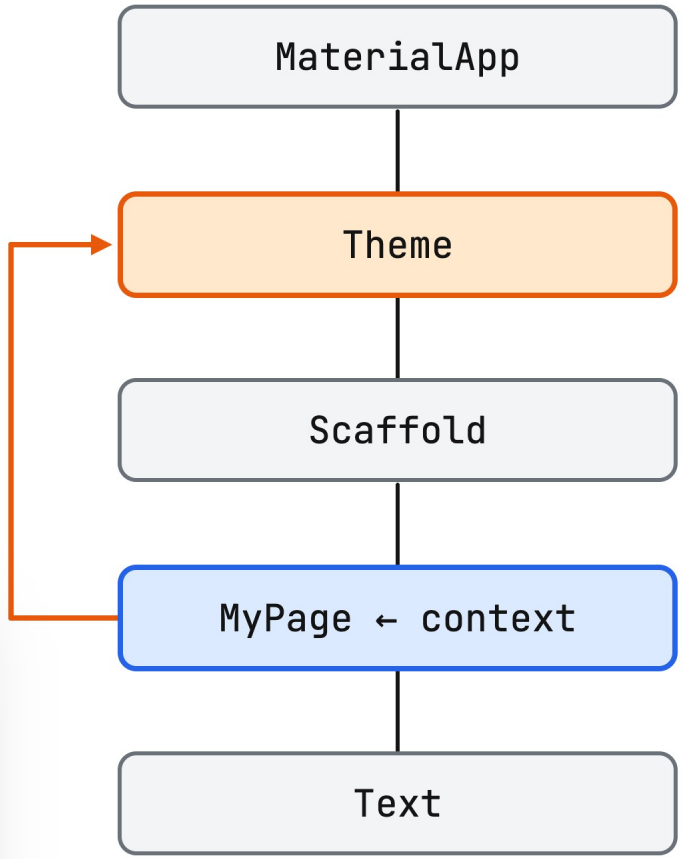
LeanCode

of(context) szuka w górę

- context = **Twój Element**, czyli Twoje miejsce w drzewie
- X.of(context) idzie od tego miejsca **w górę** i bierze najbliższy X
- dlatego liczy się, **który** context podasz

```dart
class MyPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context); // from MyPage, upwards
    return Text('Hej', style: theme.textTheme.titleLarge);
  }
}
```

Theme.of(context)
idzie w górę



We build digital products.
LeanCode

Czytanie przez context = subskrypcja (albo nie)

- Flutter (BuildContext / framework)
- pakiet provider

	context.watch<T>() / dependOn... OfExactType	context.read<T>() / getInherited... OfExactType
Co robi	czyta wartość i zapisuje Twój Element jako zależny	tylko czyta wartość, raz
Gdy wartość się zmieni	Element się przebudowuje (a State dostaje didChangeDependencies)	nic się nie dzieje
Gdzie używać	w build	w callbackach (onPressed itp.)

Theme.of, MediaQuery.of, Provider.of(context) – pod spodem wszystkie robią dependOn..., więc subskrybują.
read i Provider.of(context, listen: false) – getInherited..., bez subskrypcji.

We build digital products.
LeanCode

Pułapka #1: context „nad” providerem

context z build() to Element **MyPage**. Provider jest **pod** nim, więc szukając w górę, nigdy go nie znajdziesz.

```dart
class MyPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Provider<User>(
      create: (_) => User('Ala'),
      child: Text(context.watch<User>().name), // ❌ ProviderNotFound
    );
  }
}
```

Rozwiązanie: Builder (albo osobny widget) daje nowy context – już pod Providerem.

```dart
Provider<User>(
  create: (_) => User('Ala'),
  child: Builder(
    builder: (context) => Text(context.watch<User>().name), // ✅
  ),
)
```

Ta sama historia z Scaffold.of(context) w widgecie, który sam buduje Scaffold.

We build digital products.

LeanCode

Pułapka #2: context po await

W trakcie await użytkownik mógł już wyjść z ekranu. Element jest odmontowany, a jego context – nieważny.

Zanim użyjesz context po await, sprawdź **context.mounted** (w State wystarczy **mounted**).

Lint use_build_context_synchronously wyłapie to za Ciebie.

```dart
ElevatedButton(
  onPressed: () async {
    await api.save(form);
    // the widget might be gone by now!
    if (!context.mounted) return;
    Navigator.of(context).pop();
  },
  child: const Text('Save'),
)
```

We build digital products.

LeanCode

BuildContext w pigułce

- to **Twój Element** – Twoje miejsce w drzewie
- X.of(context) szuka **w górę** od tego miejsca
- watch / dependOn... = **subskrypcja** → rebuild, gdy wartość się zmieni
- read / getInherited... = jednorazowy odczyt, idealny w callbackach
- jest ważny tylko, dopóki Element jest zamontowany (**mounted**)

We build digital products.

LeanCode

Parę słów o obserwowaniu stanu

We build digital products.

LeanCode

Listenable

```dart
/// An object that maintains a list of listeners.
/// [...]
abstract class Listenable {
  // [...]

  /// Register a closure to be called when the object notifies its listeners.
  void addListener(VoidCallback listener);

  /// Remove a previously registered closure from the list of closures that the
  /// object notifies.
  void removeListener(VoidCallback listener);
}
```

We build digital products.

LeanCode

ChangeNotifier

```dart
class Cart extends ChangeNotifier {
  final List<Product> _items = [];

  List<Product> get items => UnmodifiableListView(_items);
  int get totalPrice => _items.fold(0, (sum, p) => sum + p.price);

  void add(Product product) {
    _items.add(product);
    notifyListeners();
  }

  void remove(Product product) {
    _items.remove(product);
    notifyListeners();
  }
}
```

Stan koszyka trzymamy **poza widgetami** – w zwykłej klasie Dart.

Każda zmiana kończy się **notifyListeners()**.

Słuchać może kilka miejsc naraz: ikonka z licznikiem w AppBarze, lista produktów, podsumowanie zamówienia.

We build digital products.

LeanCode

ValueNotifier

pudełko na **jedną wartość**

value = ... sam powiadamia słuchaczy

tylko gdy nowa wartość **!=** stara

bez n

```dart
void main(List<String> args) {
  final notifier = ValueNotifier('foobar');
  void listener() => print('changed: ${notifier.value}');

  notifier.addListener(listener);
  print(notifier.value);        // foobar

  notifier.value = 'abcdef';    // changed: abcdef
  notifier.value = 'abcdef';    // same value - no notification

  notifier.removeListener(listener);
  notifier.value = 'xyz';       // nothing printed
}
```

We build digital products.

LeanCode

Subskrypcja Listenable

```dart
class _LabelState extends State<Label> { // widget.notifier: ValueNotifier<String>
  late String _text = widget.notifier.value;
  void _update() => setState(() => _text = widget.notifier.value); ②
  @override
  void initState() {
    super.initState();
    widget.notifier.addListener(_update); ①
  }
  @override
  void didUpdateWidget(Label old) {
    super.didUpdateWidget(old);
    if (old.notifier != widget.notifier) { ③
      old.notifier.removeListener(_update);
      widget.notifier.addListener(_update);
      _text = widget.notifier.value;
    }
  }
  @override
  void dispose() {
    widget.notifier.removeListener(_update); ④
    super.dispose();
  }
  @override
  Widget build(BuildContext context) => Text(_text);
}
```

② zmiana → **setState** → rebuild

① zapisz się w **initState**

③ nowy notifier od rodzica? przepnij się w **didUpdateWidget** i odśwież **_text**

④ wypisz się w **dispose** – inaczej wyciek i błąd po usunięciu

We build digital products.

LeanCode

W praktyce: ListenableBuilder

```
ListenableBuilder(
  listenable: cart,
  builder: (context, child) ⇒ Row(children: [
    child!,
    Text('${cart.items.length}'),
  ]),
  child: const Icon(Icons.shopping_cart), // built once
)
```

```
ValueListenableBuilder<String>(
  valueListenable: label,
  builder: (context, value, child) ⇒ Text(value),
)
```

- całe ①–④ z poprzedniego slajdu – addListener, removeListener, didUpdateWidget – zrobione za Ciebie
- przebudowuje się **tylko builder**, nie cały ekran
- **child** – kawałek, który się nie zmienia: budowany raz, podawany do buildera
- dla **ValueNotifier**: ValueListenableBuilder dostaje od razu **value**

We build digital products.

LeanCode

Rodzaje Listenable

klasa	co to jest	kiedy używać
`ChangeNotifier`	klasa bazowa – sam wołasz notifyListeners()	**pełny obiekt logiki biznesowej** – własne metody, kilka pól, walidacja: koszyk, formularz, sesja
`ValueNotifier<T>`	ChangeNotifier z jednym value – przypisanie samo powiadamia	„głupie" pudełko na **jedną wartość** – tylko reagujesz na jej zmianę: licznik, wybrany tab, flaga
`Animation<T>`	wartość zmieniająca się co klatkę (AnimationController)	animacje i przejścia
`…Controller`	ScrollController, TextEditingController, TabController – ChangeNotifier / ValueNotifier pod spodem	sterowanie wbudowanymi widgetami i odczyt ich stanu
`Listenable.merge`	łączy kilka Listenable w jeden	reakcja na kilka źródeł naraz – zwłaszcza w animacjach (kilka AnimationControllerów w jednym AnimatedBuilder / CustomPainter)

Wspólny interfejs: addListener / removeListener – dlatego każdy działa z ListenableBuilder.

We build digital products.
LeanCode

Wszystko razem: ChangeNotifier + Provider

1. Wstawiamy Cart wysoko w drzewie

```
ChangeNotifierProvider(
  create: (_) => Cart(),
  child: const MyApp(),
)
```

`InheritedWidget`
dostarcza Cart w dół drzewa

`watch`
zapisuje Element jako zależny

`notifyListeners()`
przebudowuje zależne Elementy

2. Czytamy i zmieniamy gdziekolwiek poniżej

```
class CartBadge extends StatelessWidget {
  const CartBadge({super.key});

  @override
  Widget build(BuildContext context) {
    final count = context.watch<Cart>().items.length;
    return Badge(
      label: Text('$count'),
      child: const Icon(Icons.shopping_cart),
    );
  }
}

// in a callback, e.g. "Add to cart":
onPressed: () => context.read<Cart>().add(product),
```

We build digital products.
LeanCode

Dobre praktyki

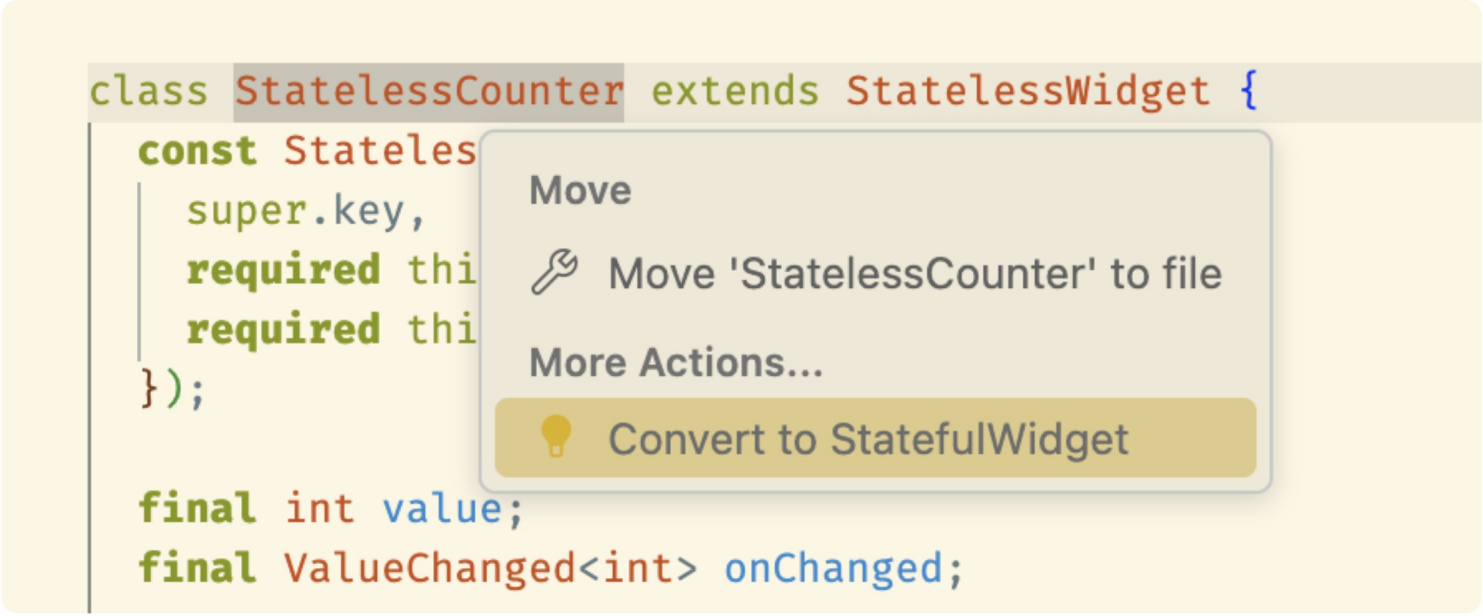
We build digital products.

LeanCode

Stateless > stateful

Bo StatelessWidget jest:

- tańszy – brak obiektu stanu (różnica minimalna, ale w dużej skali może coś dać)
- krótszy
- szybciej się go pisze
- a jak trzeba, VS Code przerobi stateless ←→ stateful jednym kliknięciem:



We build digital products.
LeanCode

const, gdzie się da

```dart
Column(children: [
  const Header(), // same instance on every build
  Text('Hej $name'), // new instance on every build
])
```

- const widget to **ta sama instancja** przy każdym buildzie rodzica
- Flutter sprawdza najpierw identical(A, B) → prawda → **całe poddrzewo pominięte**
- dlatego warto mieć **const konstruktory** we własnych widgetach
- lint prefer_const_constructors ma **autofix** – zapisujesz plik i const dopisuje się sam

We build digital products.

LeanCode

Małe widgety zamiast metod _buildX()

❌ **metoda – ten sam context co strona**

```
class ProfilePage extends StatelessWidget {
  // ...
  Widget _buildHeader(BuildContext context) {
    final theme = Theme.of(context);
    return Text('Profile',
        style: theme.textTheme.titleLarge);
  }

  @override
  Widget build(BuildContext context) ⇒
      Column(children: [_buildHeader(context)]);
}
```

✅ **widget – własny Element i context**

```
class _Header extends StatelessWidget {
  const _Header();

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return Text('Profile',
        style: theme.textTheme.titleLarge);
  }
}

// in ProfilePage.build:
Column(children: [const _Header()])
```

- Theme.of subskrybuje **tylko _Header**, a nie całą stronę – przebudowa obejmuje mniej
- może być **const** → pomijany przy rebuildzie rodzica
- własny context = koniec z pułapką „context nad providerem”, a w DevTools widać osobny węzeł

We build digital products.

LeanCode

build() – szybki i bez efektów ubocznych

- build może się odpalić **w każdej klatce** – setState, animacja, zmiana MediaQuery, rebuild rodzica
- więc w build **nie**: wołamy API, tworzymy controllerów, odpalamy Future, zapisujemy stanu
- to wszystko ląduje w **State**: initState, callbacki, dispose

```
// ❌ new request on every rebuild
FutureBuilder(future: api.fetchUser(), builder: /* ... */);

// ✅ create the Future once, in State
late final _user = api.fetchUser();
// ...
FutureBuilder(future: _user, builder: /* ... */);
```

We build digital products.

LeanCode

Nie nadużywaj Providera/InheritedWidgeta

Jak wartości przychodzą „z powietrza” przez InheritedWidget albo Providera, to osoba czytająca kod (albo robiąca review) łatwo się pogubi.

Tracisz też pomoc analizy statycznej, którą masz przy zwykłym przekazywaniu parametrów – błąd wyjdzie dopiero w runtime.

We build digital products.

LeanCode

Trwałe obiekty trzymaj w State, nie w widgecie

❌ pole w StatelessWidget

```dart
class SearchField extends StatelessWidget {
  SearchField({super.key});

  // new controller on every parent rebuild!
  final controller = TextEditingController();

  @override
  Widget build(BuildContext context) ⇒
      TextField(controller: controller);
}
```

✅ pole w State + dispose

```dart
class _SearchFieldState extends State<SearchField> {
  final _controller = TextEditingController();

  @override
  void dispose() {
    _controller.dispose();
    super.dispose();
  }

  @override
  Widget build(BuildContext context) ⇒
      TextField(controller: _controller);
}
```

Widget żyje do następnego rebuildu rodzica – nowa instancja = nowy controller: tekst znika, a stary nigdy nie dostaje **dispose()**.
Dotyczy też FocusNode, ScrollController, AnimationController, subskrypcji...

We build digital products.
LeanCode

Trzy drzewa w pigułce

Widgety
lekkie i niezmienne

- **niemutowalna konfiguracja** dla elementów
- **przepis** na fragment UI
- nie wiedzą, gdzie są w drzewie
- tanie i jednorazowe – nowe przy każdym buildzie
- jedna instancja może skonfigurować **wiele** elementów

Elementy
trwałe i mutowalne

- **konkretne miejsce** w drzewie – to jest BuildContext
- **przeżywają rebuildy** – dostają update(newWidget)
- mają cykl życia
- znają przodków i potomków
- trzymają swój widget i jego **State**

RenderObjecty
layout · paint · dostępność

- **layout** – rozmiar i pozycja
- **painting** – rysują bezpośrednio na canvasie
- **semantyka** – to, co widzi czytnik ekranu
- biorą udział w **hit testingu**
- na co dzień ich nie piszesz – bierz gotowe widgety

Widget mówi co, Element pamięta gdzie, RenderObject robi jak.

We build digital products.
LeanCode

Polecajki do czytania

- Flutter architectural overview
- ↳ O stanie i inherited widgetach
- ↳ O elementach i render objectach
- O algorytmie rekoncyliacji widgetów
- Key class - foundation library - Dart API (flutter.dev)
- flutter_hooks | Flutter Package – inne podejście do stanu, zbudowane na StatefulWidget
- boxy | Flutter package – łatwiejsze własne layouty (bez pisania własnych RenderObjectów)

We build digital products.

LeanCode